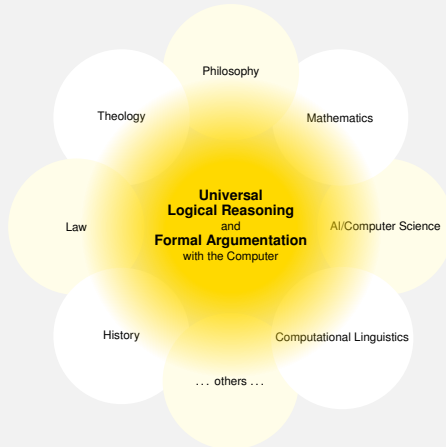


Universal Logical Reasoning and Formal Argumentation

Christoph Benz Müller

Freie Universität Berlin & Latentine/Tidy



TU Berlin, 24 Feb 2021

Universal Logical Reasoning and Formal Argumentation

Christoph Benz Müller

Freie Universität Berlin & Latentine/Tidy

*“If we had it [a **characteristica universalis**], we should be able to reason in metaphysics and morals in much the same way as in geometry and analysis.”*

(Leibniz, 1690; translation by Russell)

TU Berlin, 24 Feb 2021

What I could – but won't – talk about ...



Hundreds of
media
reports
worldwide

“Formal Arguments in Metaphysics”

What I could – but won't – talk about ...

- ▶ **Simplified Variant of Gödel's Ontological Argument based on Ultrafilter-Theory**



Recent talk: <https://dx.doi.org/10.13140/RG.2.2.10224.69129>

- ▶ **Discussion (for wider public) of my Contributions to Computational Metaphysics**



What I could – but won't – talk about ...



Ed Zalta (Stanford)

Principia Logico-Metaphysica

Hyperintensional higher-order modal logic

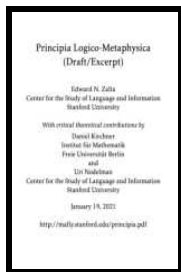
Inconsistency/Paradox detected & fixed

[Open Philosophy, 2019] [RSL, 2020]



Daniel Kirchner
(Mathematics, FU Berlin)

What I could – but won't – talk about ...



Ed Zalta (Stanford)

Principia Logico-Metaphysica

Hyperintensional higher-order modal logic

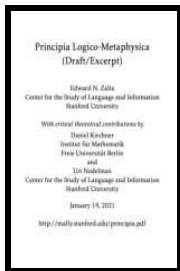
Inconsistency/Paradox detected & fixed

[Open Philosophy, 2019] [RSL, 2020]



Daniel Kirchner
(Mathematics, FU Berlin)

What I could – but won't – talk about ...



Ed Zalta (Stanford)

Principia Logico-Metaphysica

Hyperintensional higher-order modal logic

Inconsistency/Paradox detected & fixed

[Open Philosophy, 2019] [RSL, 2020]



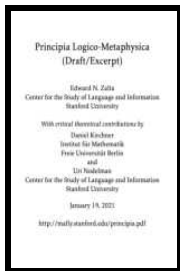
Daniel Kirchner
(Mathematics, FU Berlin)

Kirchner Paradox

Daniel & Isabelle/HOL have become close advisors of
Ed Zalta in the search for a repair

Computational Metaphysics par excellence!!!

What I could – but won't – talk about ...



Ed Zalta (Stanford)

Principia Logico-Metaphysica

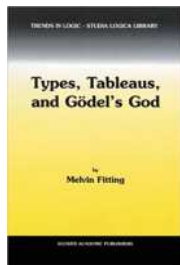
Hyperintensional higher-order modal logic

Inconsistency/Paradox detected & fixed

[Open Philosophy, 2019] [RSL, 2020]



Daniel Kirchner
(Mathematics, FU Berlin)



M. Fitting (New York)

Philosophy/Metaphysics

Ontological Argument
(avoids modal collapse)

Intensional higher-order modal logic

Verified (main chapters)

[Archive of Formal Proofs, 2017]



David Fuenmayor
(Philosophy, FU Berlin)

What I could – but won't – talk about ...



E. J. Lowe's (Durham)

Philosophy/Metaphysics

E. J. Lowe's Modal Ontological Argument

Higher-order modal logic

Verified (minor corrections)

[Journal of Applied Logics, 2018]



David Fuenmayor
(Philosophy, FU Berlin)

What I could – but won't – talk about ...



E. J. Lowe's (Durham)

Philosophy/Metaphysics

E. J. Lowe's Modal Ontological Argument

Higher-order modal logic

Verified (minor corrections)

[Journal of Applied Logics, 2018]



David Fuenmayor
(Philosophy, FU Berlin)



P. Freyd & A. Scedrov

Category Theory

Free first-order logic

(Constricted) Inconsistency detected & fixed

[JAR, 2019]

Lucca Tiemens, Irina Makaranko, Aleksey Gonus, Natalia Konstantinova, Jonas Bayer



D. Scott
(UC Berkeley)

What I will talk about (in the next 25min):

1. Motivation: Trustworthy AI (How to control?)
2. Higher-Order Logic
3. Universal Logical Reasoning
4. LogiKEY Methodology and Infrastructure
5. Application to Legal Balancing



1. Trustworthy AI (How to control?)

Complex Technology

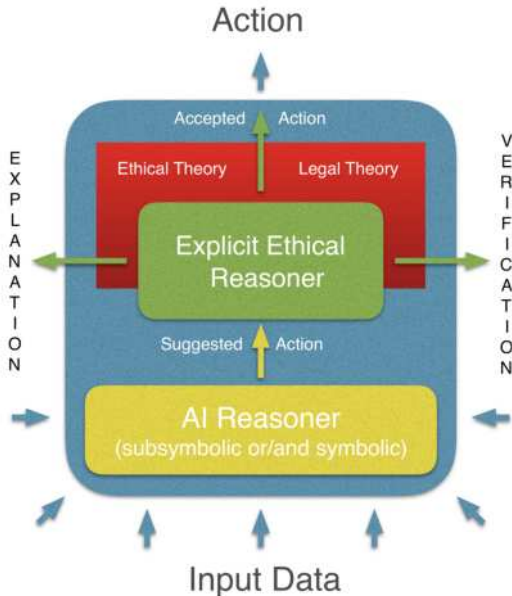


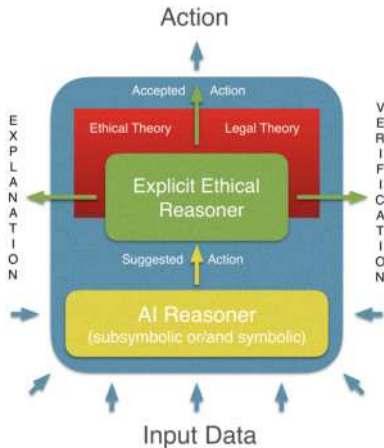
Complex Technology — Increasingly AI-Driven



Things Go Wrong — Is Explaining already Enough?







Related Works

- ▶ Toward Ethical Robots
 - ▶ [ArkoudasEtAl., 2005]
- ▶ Artificial Moral Agents
 - ▶ [Wallach&Allen, 2008]
- ▶ Ethical Governors
 - ▶ [ArkinEtAl., 2009, 2012]
 - ▶ [Dennis&Fisher, 2017]
- ▶ Ethical Deliberation in ART
 - ▶ [Dignum, 2017]
- ▶ Programming Machine Ethics
 - ▶ [Pereira&Saptawijaya, 2016]

Addresses demands for explainability and verifiability:

⇒ but not at the level of AI black box systems!

[German Conference on Artificial Intelligence \(Künstliche Intelligenz\)](#)

..... KI 2020: [KI 2020: Advances in Artificial Intelligence](#) pp 251-258 | [Cite as](#)

Reasonable Machines: A Research Manifesto

Authors

[Authors and affiliations](#)

Christoph Benz Müller , Bertram Lomfeld

**Building trust into AI systems through
rational communication of »reasons«**



2. Higher-Order Logic

Higher-Order Logic (HOL)

Expressivity	FOL	HOL	Example
Quantification over			
- Individuals	✓	✓	$\forall X p(f(X))$
- Functions	-	✓	$\forall F p(F(a))$
- Predicates/Sets/Rels	-	✓	$\forall P P(f(a))$
Unnamed			
- Functions	-	✓	$\lambda X X$
- Predicates	-	✓	$\lambda X (X = X)$
Statements about			
- Funcs/Preds/Sets/Rels	-	✓	<i>reflexive</i> =
Powerful definitions	-	✓	<i>reflexive</i> := $\lambda R \forall X (R X X)$

Higher-Order Logic (HOL)

Expressivity	FOL	HOL	Example
Quantification over			
- Individuals	✓	✓	$\forall X p(f(X))$
- Functions	-	✓	$\forall F p(F(a))$
- Predicates/Sets/Rels	-	✓	$\forall P P(f(a))$
Unnamed			
- Functions	-	✓	$\lambda X X$
- Predicates	-	✓	$\lambda X (X = X)$
Statements about			
- Funcs/Preds/Sets/Rels	-	✓	<i>reflexive</i> =
Powerful definitions	-	✓	<i>reflexive</i> := $\lambda R \forall X (R X X)$

Higher-Order Logic (HOL)

Expressivity	FOL	HOL	Example
Quantification over			
- Individuals	✓	✓	$\forall X_i p_{i \rightarrow o}(f_{i \rightarrow i}(X_i))$
- Functions	-	✓	$\forall F_{i \rightarrow i} p_{i \rightarrow o}(F_{i \rightarrow o}(a_i))$
- Predicates/Sets/Rels	-	✓	$\forall P_{i \rightarrow o} P_{i \rightarrow o}(f_{i \rightarrow i}(a_i))$
Unnamed			
- Functions	-	✓	$\lambda X_i X$
- Predicates	-	✓	$\lambda X_i (X = X)$
Statements about			
- Funcs/Preds/Sets/Rels	-	✓	$reflexive_{(o \rightarrow o \rightarrow o) \rightarrow o} =_{o \rightarrow o \rightarrow o}$
Powerful definitions	-	✓	$reflexive_{(\alpha \rightarrow \alpha \rightarrow o) \rightarrow o} :=$ $\lambda R_{(\alpha \rightarrow \alpha \rightarrow o)} \forall X_\alpha (R X X)$

Types: Prevent paradoxes and inconsistencies

Higher-Order Logic (HOL)

Simple Types:

$$\alpha, \beta ::= i \mid o \mid (\alpha \rightarrow \beta)$$

Simply Typed λ -Calculus (with constants):

$$s, t ::= p_\alpha \mid X_\alpha \mid (\lambda X_\alpha s_\beta)_{\alpha \rightarrow \beta} \mid (s_{\alpha \rightarrow \beta} t_\alpha)_\beta$$

constants variables lambda abstraction application

abstraction and application interact, e.g.: $((\lambda X (p X)) t) \xrightarrow{\beta\text{-reduction}} (p t)$

HOL defined on Top of Simply Typed λ -Calculus

- ▶ add special constant symbols to signature, e.g.

$$\neg_{o \rightarrow o} \quad \forall_{o \rightarrow o \rightarrow o} \quad \Pi_{(o \rightarrow o) \rightarrow o}$$

- ▶ no binder besides λ needed: $\forall X_\alpha s_o$ stands for $\Pi_{(o \rightarrow o) \rightarrow o} (\lambda X_\alpha s_o)$
- ▶ other connectives can now be defined

HOL is a language of terms. Terms of type o are called formulas.

HOL has a well understood and well documented semantics.

Higher-Order Logic (HOL)

Simple Types:

$$\alpha, \beta ::= i \mid o \mid (\alpha \rightarrow \beta)$$

Simply Typed λ -Calculus (with constants):

$$s, t ::= p_\alpha \mid X_\alpha \mid (\lambda X_\alpha s_\beta)_{\alpha \rightarrow \beta} \mid (s_{\alpha \rightarrow \beta} t_\alpha)_\beta$$

constants variables lambda abstraction application

abstraction and application interact, e.g.: $((\lambda X (p X)) t) \xrightarrow{\beta\text{-reduction}} (p t)$

HOL defined on Top of Simply Typed λ -Calculus

- ▶ add special constant symbols to signature, e.g.

$$\neg_{o \rightarrow o} \quad \vee_{o \rightarrow o \rightarrow o} \quad \Pi_{(\alpha \rightarrow o) \rightarrow o}$$

- ▶ no binder besides λ needed: $\forall X_\alpha s_o$ stands for $\Pi_{(\alpha \rightarrow o) \rightarrow o} (\lambda X_\alpha s_o)$
- ▶ other connectives can now be defined

HOL is a language of terms. Terms of type o are called formulas.

HOL has a well understood and well documented semantics.

Higher-Order Logic (HOL)

Simple Types:

$$\alpha, \beta ::= i \mid o \mid (\alpha \rightarrow \beta)$$

Simply Typed λ -Calculus (with constants):

$$s, t ::= \boxed{p_\alpha} \mid X_\alpha \mid (\lambda X_\alpha s_\beta)_{\alpha \rightarrow \beta} \mid (s_{\alpha \rightarrow \beta} t_\alpha)_\beta$$

constants variables lambda abstraction application

abstraction and application interact, e.g.: $((\lambda X (p X)) t) \xrightarrow{\beta\text{-reduction}} (p t)$

HOL defined on Top of Simply Typed λ -Calculus

- ▶ add special constant symbols to signature, e.g.

$$\neg_{o \rightarrow o} \quad \vee_{o \rightarrow o \rightarrow o} \quad \Pi_{(\alpha \rightarrow o) \rightarrow o}$$

- ▶ no binder besides λ needed: $\forall X_\alpha s_o$ stands for $\Pi_{(\alpha \rightarrow o) \rightarrow o} (\lambda X_\alpha s_o)$
- ▶ other connectives can now be defined

HOL is a language of terms. Terms of type o are called formulas.

HOL has a well understood and well documented semantics.

Higher-Order Logic (HOL)

Simple Types:

$$\alpha, \beta ::= i \mid o \mid (\alpha \rightarrow \beta)$$

Simply Typed λ -Calculus (with constants):

$$s, t ::= p_\alpha \mid \boxed{X_\alpha} \mid (\lambda X_\alpha s)_\beta \mid (s_{\alpha \rightarrow \beta} t_\alpha)_\beta$$

constants variables lambda abstraction application

abstraction and application interact, e.g.: $((\lambda X (p X)) t) \xrightarrow{\beta\text{-reduction}} (p t)$

HOL defined on Top of Simply Typed λ -Calculus

- ▶ add special constant symbols to signature, e.g.

$$\neg_{o \rightarrow o} \quad \forall_{o \rightarrow o \rightarrow o} \quad \Pi_{(\alpha \rightarrow o) \rightarrow o}$$

- ▶ no binder besides λ needed: $\forall X_\alpha s_o$ stands for $\Pi_{(\alpha \rightarrow o) \rightarrow o} (\lambda X_\alpha s_o)$
- ▶ other connectives can now be defined

HOL is a language of terms. Terms of type o are called formulas.

HOL has a well understood and well documented semantics.

Higher-Order Logic (HOL)

Simple Types:

$$\alpha, \beta ::= i \mid o \mid (\alpha \rightarrow \beta)$$

Simply Typed λ -Calculus (with constants):

$$s, t ::= p_\alpha \mid X_\alpha \mid (\lambda X_\alpha s_\beta)_{\alpha \rightarrow \beta} \mid (s_{\alpha \rightarrow \beta} t_\alpha)_\beta$$

constants variables lambda abstraction application

abstraction and application interact, e.g.: $((\lambda X (p X)) t) \xrightarrow{\beta\text{-reduction}} (p t)$

HOL defined on Top of Simply Typed λ -Calculus

- ▶ add special constant symbols to signature, e.g.

$$\neg_{o \rightarrow o} \quad \vee_{o \rightarrow o \rightarrow o} \quad \Pi_{(\alpha \rightarrow o) \rightarrow o}$$

- ▶ no binder besides λ needed: $\forall X_\alpha s_o$ stands for $\Pi_{(\alpha \rightarrow o) \rightarrow o} (\lambda X_\alpha s_o)$
- ▶ other connectives can now be defined

HOL is a language of terms. Terms of type o are called formulas.

HOL has a well understood and well documented semantics.

Higher-Order Logic (HOL)

Simple Types:

$$\alpha, \beta ::= i \mid o \mid (\alpha \rightarrow \beta)$$

Simply Typed λ -Calculus (with constants):

$$s, t ::= p_\alpha \mid X_\alpha \mid (\lambda X_\alpha s_\beta)_{\alpha \rightarrow \beta} \mid \boxed{(s_{\alpha \rightarrow \beta} t_\alpha)_\beta}$$

constants variables lambda abstraction application

abstraction and application interact, e.g.: $((\lambda X (p X)) t) \xrightarrow{\beta\text{-reduction}} (p t)$

HOL defined on Top of Simply Typed λ -Calculus

- ▶ add special constant symbols to signature, e.g.

$$\neg_{o \rightarrow o} \quad \vee_{o \rightarrow o \rightarrow o} \quad \Pi_{(\alpha \rightarrow o) \rightarrow o}$$

- ▶ no binder besides λ needed: $\forall X_\alpha s_o$ stands for $\Pi_{(\alpha \rightarrow o) \rightarrow o} (\lambda X_\alpha s_o)$
- ▶ other connectives can now be defined

HOL is a language of terms. Terms of type o are called formulas.

HOL has a well understood and well documented semantics.

Higher-Order Logic (HOL)

Simple Types:

$$\alpha, \beta ::= i \mid o \mid (\alpha \rightarrow \beta)$$

Simply Typed λ -Calculus (with constants):

$$s, t ::= p_\alpha \mid X_\alpha \mid (\lambda X_\alpha s_\beta)_{\alpha \rightarrow \beta} \mid (s_{\alpha \rightarrow \beta} t_\alpha)_\beta$$

constants variables lambda abstraction application

abstraction and application interact, e.g.: $((\lambda X (p X)) t) \xrightarrow{\beta\text{-reduction}} (p t)$

HOL defined on Top of Simply Typed λ -Calculus

- ▶ add special constant symbols to signature, e.g.

$$\neg_{o \rightarrow o} \quad \forall_{o \rightarrow o \rightarrow o} \quad \Pi_{(\alpha \rightarrow o) \rightarrow o}$$

- ▶ no binder besides λ needed: $\forall X_\alpha s_o$ stands for $\Pi_{(\alpha \rightarrow o) \rightarrow o} (\lambda X_\alpha s_o)$
- ▶ other connectives can now be defined

HOL is a language of terms. Terms of type o are called formulas.

HOL has a well understood and well documented semantics.

Higher-Order Logic (HOL)

Simple Types:

$$\alpha, \beta ::= i \mid o \mid (\alpha \rightarrow \beta)$$

Simply Typed λ -Calculus (with constants):

$$s, t ::= p_\alpha \mid X_\alpha \mid (\lambda X_\alpha s_\beta)_{\alpha \rightarrow \beta} \mid (s_{\alpha \rightarrow \beta} t_\alpha)_\beta$$

constants variables lambda abstraction application

abstraction and application interact, e.g.: $((\lambda X (p X)) t) \xrightarrow{\beta\text{-reduction}} (p t)$

HOL defined on Top of Simply Typed λ -Calculus

- ▶ add special constant symbols to signature, e.g.

$$\neg_{o \rightarrow o} \quad \forall_{o \rightarrow o \rightarrow o} \quad \Pi_{(\alpha \rightarrow o) \rightarrow o}$$

- ▶ no binder besides λ needed: $\forall X_\alpha s_o$ stands for $\Pi_{(\alpha \rightarrow o) \rightarrow o} (\lambda X_\alpha s_o)$
- ▶ other connectives can now be defined

HOL is a language of terms. Terms of type o are called formulas.

HOL has a well understood and well documented semantics.

Higher-Order Logic (HOL)

Simple Types:

$$\alpha, \beta ::= i \mid o \mid (\alpha \rightarrow \beta)$$

Simply Typed λ -Calculus (with constants):

$$s, t ::= p_\alpha \mid X_\alpha \mid (\lambda X_\alpha s_\beta)_{\alpha \rightarrow \beta} \mid (s_{\alpha \rightarrow \beta} t_\alpha)_\beta$$

constants variables lambda abstraction application

abstraction and application interact, e.g.: $((\lambda X (p X)) t) \xrightarrow{\beta\text{-reduction}} (p t)$

HOL defined on Top of Simply Typed λ -Calculus

- ▶ add special constant symbols to signature, e.g.

$$\neg_{o \rightarrow o} \quad \vee_{o \rightarrow o \rightarrow o} \quad \Pi_{(\alpha \rightarrow o) \rightarrow o}$$

- ▶ no binder besides λ needed: $\forall X_\alpha s_o$ stands for $\Pi_{(\alpha \rightarrow o) \rightarrow o} (\lambda X_\alpha s_o)$
- ▶ other connectives can now be defined

HOL is a language of terms. Terms of type o are called formulas.

HOL has a well understood and well documented semantics.

Higher-Order Logic (HOL)

Simple Types:

$$\alpha, \beta ::= i \mid o \mid (\alpha \rightarrow \beta)$$

Simply Typed λ -Calculus (with constants):

$$s, t ::= p_\alpha \mid X_\alpha \mid (\lambda X_\alpha s_\beta)_{\alpha \rightarrow \beta} \mid (s_{\alpha \rightarrow \beta} t_\alpha)_\beta$$

constants variables lambda abstraction application

abstraction and application interact, e.g.: $((\lambda X (p X)) t) \xrightarrow{\beta\text{-reduction}} (p t)$

HOL defined on Top of Simply Typed λ -Calculus

- ▶ add special constant symbols to signature, e.g.

$$\neg_{o \rightarrow o} \quad \forall_{o \rightarrow o \rightarrow o} \quad \Pi_{(\alpha \rightarrow o) \rightarrow o}$$

- ▶ no binder besides λ needed: $\forall X_\alpha s_o$ stands for $\Pi_{(\alpha \rightarrow o) \rightarrow o} (\lambda X_\alpha s_o)$
- ▶ other connectives can now be defined

HOL is a language of terms. Terms of type o are called formulas.

HOL has a well understood and well documented semantics.

HOL: Theorem Provers

Interactive Systems: Isabelle, HOL4, HOL light, PVS, IMPS, Coq, Lean, ...

TPTP Infrastructure for proof automation (since 2008) (jww Sutcliffe & others)

This fostered the developed of Automated Theorem Provers for HOL:

TPS	CMU, US	1980s–2000s	Mating Method
LEO-I	U Saarbrücken, DE	1995–2006	Resolution
LEO-II	Cambridge U, UK	2007–	Paramodulation
Satallax	Saarbr./Paris-Saclay, FR	2010–	Tableaux
IsabelleP	Isabelle Team, DE/UK/NL	2010–	Misc
Leo-III	Berlin/Luxemb., DE/LU	2013–	Ordered Paramodulation
CoqATP	Coq Team, FR	2013–	Saturation Algorithm
agsyHOL	Chalmers, SE	2014–	Lazy Narrowing
HOLyHammer	Innsbr./Praha, AU/CZ	2014–	Misc
Zipperpin	Vrije U Amsterdam, NL	2017–	Superposition
CVC4	U Iowa, US	2019–	SMT Solver
Vampire	U Manchester, UK	2020–	Superposition
...	...	...	...

Model finders:

Nitpick, Nunchaku

Survey articles:

[BenzmüllerMiller, Automation of Higher-Order Logic, HB of History Of Logic, 2014]

[BenzmüllerAndrews, Church's Type Theory, SEP, 2019]

Why Classical Logic is not enough

The Problem with Modalities

Peter **knows** “New York is a city in the US”

“New York is a city in the US”

True

True

“Batman is a city in Turkey”

Peter **knows** “Batman is a city in Turkey”

Need for Modal Logics: ☐ Belief ☐ Time ☐ Obligation ☐ ...

Many other Non-Classical Logics (besides Modal Logics)

In general: combinations of non-classical logics are needed

Challenge Question: **How to Tame the Expanding Logic Zoo?**

Why Classical Logic is not enough

The Problem with Modalities

Peter **knows** “New York is a city in the US”

“New York is a city in the US”

True

True

“Batman is a city in Turkey”

Peter knows “Batman is a city in Turkey”

Need for Modal Logics: ☐ Belief ☐ Time ☐ Obligation ☐ ...

Many other Non-Classical Logics (besides Modal Logics)

In general: combinations of non-classical logics are needed

Challenge Question: **How to Tame the Expanding Logic Zoo?**

Why Classical Logic is not enough

The Problem with Modalities

Peter **knows** “New York is a city in the US”

“New York is a city in the US”

True

True

“Batman is a city in Turkey”

Peter knows “Batman is a city in Turkey”

Need for Modal Logics: ☐ Belief ☐ Time ☐ Obligation ☐ ...

Many other Non-Classical Logics (besides Modal Logics)

In general: combinations of non-classical logics are needed

Challenge Question: How to Tame the Expanding Logic Zoo?

Why Classical Logic is not enough

The Problem with Modalities

Peter **knows** “New York is a city in the US”

“New York is a city in the US”

True

$\leftrightarrow$

True

“Batman is a city in Turkey”

Peter knows “Batman is a city in Turkey”

Need for Modal Logics: ☐ Belief ☐ Time ☐ Obligation ☐ ...

Many other Non-Classical Logics (besides Modal Logics)

In general: combinations of non-classical logics are needed

Challenge Question: How to Tame the Expanding Logic Zoo?

Why Classical Logic is not enough

The Problem with Modalities

Peter **knows** “New York is a city in the US”

“New York is a city in the US”

True

=

True

“Batman is a city in Turkey”

Peter knows “Batman is a city in Turkey”

Need for Modal Logics: ☐ Belief ☐ Time ☐ Obligation ☐ ...

Many other Non-Classical Logics (besides Modal Logics)

In general: combinations of non-classical logics are needed

Challenge Question: How to Tame the Expanding Logic Zoo?

Why Classical Logic is not enough

The Problem with Modalities

Peter **knows** “New York is a city in the US”

“New York is a city in the US”

True

=

True

“Batman is a city in Turkey”

Peter **knows** “Batman is a city in Turkey”

Need for Modal Logics: ☐ Belief ☐ Time ☐ Obligation ☐ ...

Many other Non-Classical Logics (besides Modal Logics)

In general: combinations of non-classical logics are needed

Challenge Question: How to Tame the Expanding Logic Zoo?

Why Classical Logic is not enough

The Problem with Modalities

Peter **knows** “New York is a city in the US”

“New York is a city in the US”

True

=

True

“Batman is a city in Turkey”

Peter **knows** “Batman is a city in Turkey”

Need for Modal Logics: ☐ Belief ☐ Time ☐ Obligation ☐ ...

Many other Non-Classical Logics (besides Modal Logics)

In general: combinations of non-classical logics are needed

Challenge Question: How to Tame the Expanding Logic Zoo?

Why Classical Logic is not enough

The Problem with Modalities

Peter **knows** “New York is a city in the US”

“New York is a city in the US”

True

=

True

“Batman is a city in Turkey”

Peter **knows** “Batman is a city in Turkey”

Need for Modal Logics: ☐ Belief ☐ Time ☐ Obligation ☐ ...

Many other Non-Classical Logics (besides Modal Logics)

In general: combinations of non-classical logics are needed

Challenge Question: How to Tame the Expanding Logic Zoo?

Why Classical Logic is not enough

The Problem with Modalities

Peter **knows** “New York is a city in the US”

“New York is a city in the US”

True

=

True

“Batman is a city in Turkey”

Peter **knows** “Batman is a city in Turkey”

Need for Modal Logics: ☐ Belief ☐ Time ☐ Obligation ☐ ...

Many other Non-Classical Logics (besides Modal Logics)

In general: combinations of non-classical logics are needed

Challenge Question: How to Tame the Expanding Logic Zoo?



3. Universal Logical Reasoning (How to Tame the Logic Zoo?)



STUDIES IN LOGIC AND PRACTICAL REASONING

VOLUME 3

D.M. GABBAY / P. GARDENFORS / J. SIEKMANN / J. VAN BENTHEM / M. VARDI / J. WOODS

EDITORS

Handbook of Modal Logic



STUDIES IN LOGIC

2 BASIC MODAL LOGIC

In this section we introduce the basic modal language and its relational semantics. We define basic modal syntax, introduce models and frames, and give the satisfaction definition. We then draw the reader's attention to the internal perspective that modal languages offer on relational structure, and explain why models and frames should be thought of as graphs. Following this we give the standard translation. This enables us to convert any basic modal formula into a first-order formula with one free variable. The standard translation is a bridge between the modal and classical worlds, a bridge that underlies much of the work of this chapter.

2.1 First steps in relational semantics

Suppose we have a set of proposition symbols (whose elements we typically write as p, q, r and so on) and a set of modality symbols (whose elements we typically write as m, m', m'' , and so on). The choice of PROP and MOD is called the *signature* (or *similarity type*) of the language; in what follows we'll tacitly assume that PROP is denumerably infinite, and we'll often work with signatures in which MOD contains only a single element. Given a signature, we define the *basic modal language* (over the signature) as follows:

$$\varphi ::= p \mid \top \mid \perp \mid \neg\varphi \mid \varphi \wedge \psi \mid \varphi \vee \psi \mid \varphi \rightarrow \psi \mid \varphi \leftrightarrow \psi \mid \langle m \rangle \varphi \mid [m] \varphi.$$

That is, a basic modal formula is either a proposition symbol, a boolean constant, a boolean combination of basic modal formulas, or (most interesting of all) a formula prefixed by a diamond

WOODS

Modal Logic

STUDIES IN LOGIC

2 BASIC MODAL LOGIC

In this section we introduce the basic modal language and its relational semantics. We define basic modal syntax, introduce models and frames, and give the satisfaction definition. We then draw the reader's attention to the internal perspective that modal languages offer on relational structure, and explain why models and frames should be thought of as graphs. Following this we give the standard translation. This enables us to convert any basic modal formula into a first-order formula with one free variable. The standard translation is a bridge between the modal and classical worlds, a bridge that underlies much of the work of this chapter.

2.1 First steps in relational semantics

Syntax

Metalanguage

What follows we usually assume that PROP is countably infinite, and we'll often work with signatures in which MOD contains only a single element. Given a signature, we define the *basic modal language* (over the signature) as follows:

$$\varphi ::= p \mid \top \mid \perp \mid \neg\varphi \mid \varphi \wedge \psi \mid \varphi \vee \psi \mid \varphi \rightarrow \psi \mid \varphi \leftrightarrow \psi \mid \langle m \rangle \varphi \mid [m] \varphi.$$

That is, a basic modal formula is either a proposition symbol, a boolean constant, a boolean combination of basic modal formulas, or (most interesting of all) a formula prefixed by a diamond

Modal Logic



STUDIES IN LOGIC

A *model* (or *Kripke model*) $\mathfrak{M}$ for the basic modal language (over some fixed signature) is a triple $\mathfrak{M} = (W, \{R^m\}_{m \in \text{MOD}}, V)$. Here W , the *domain*, is a non-empty set, whose elements we usually call *points*, but which, for reasons which will soon be clear, are sometimes called *states*, *times*, *situations*, *worlds* and other things besides. Each R^m in a model is a binary relation on W , and V is a function (the valuation) that assigns to each proposition symbol p in PROP a subset $V(p)$ of W ; think of $V(p)$ as the set of points in $\mathfrak{M}$ where p is true. The first two components $(W, \{R^m\}_{m \in \text{MOD}})$ of $\mathfrak{M}$ are called the *frame* underlying the model. If there is only one relation in the model, we typically write (W, R) for its frame, and (W, R, V) for the model itself. We encourage the reader to think of Kripke models as graphs (or to be slightly more precise, *directed graphs*, that is, graphs whose points are linked by directed arrows) and will shortly give some examples which show why this is helpful.

Suppose w is a point in a model $\mathfrak{M} = (W, \{R^m\}_{m \in \text{MOD}}, V)$. Then we inductively define the notion of a formula φ being *satisfied* (or *true*) in $\mathfrak{M}$ at point w as follows (we omit some of the clauses for the booleans):

$\mathfrak{M}, w \models p$	iff	$w \in V(p)$,
$\mathfrak{M}, w \models \top$		always,
$\mathfrak{M}, w \models \perp$		never,
$\mathfrak{M}, w \models \neg\varphi$	iff	not $\mathfrak{M}, w \models \varphi$ (notation: $\mathfrak{M}, w \not\models \varphi$),
$\mathfrak{M}, w \models \varphi \wedge \psi$	iff	$\mathfrak{M}, w \models \varphi$ and $\mathfrak{M}, w \models \psi$,
$\mathfrak{M}, w \models \varphi \rightarrow \psi$	iff	$\mathfrak{M}, w \not\models \varphi$ or $\mathfrak{M}, w \models \psi$,
$\mathfrak{M}, w \models \langle m \rangle \varphi$	iff	for some $v \in W$ such that $R^m wv$ we have $\mathfrak{M}, v \models \varphi$,
$\mathfrak{M}, w \models [m] \varphi$	iff	for all $v \in W$ such that $R^m wv$ we have $\mathfrak{M}, v \models \varphi$.



STUDIES IN LOGIC

A *model* (or *Kripke model*) $\mathfrak{M}$ for the basic modal language (over some fixed signature) is a triple $\mathfrak{M} = (W, \{R^m\}_{m \in \text{MOD}}, V)$. Here W , the *domain*, is a non-empty set, whose elements we usually call *points*, but which, for reasons which will soon be clear, are sometimes called *states*, *times*

and $V(p)$
 $(W, \{$
in the

Metalanguage

in a model is a binary relation on W .
position symbol p in PROP a subset
 p is true. The first two components
; model. If there is only one relation
 (W, R, V) for the model itself. We
encourage the reader to think of Kripke models as graphs (or to be slightly more precise, *directed graphs*, that is, graphs whose points are linked by directed arrows) and will shortly give some examples which show why this is helpful.

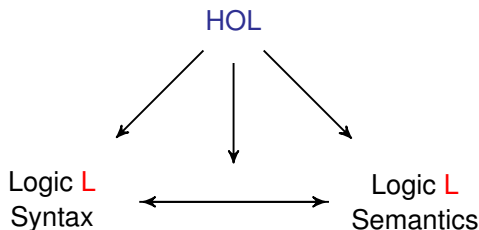
Suppose w is a point in a model $\mathfrak{M} = (W, \{R^m\}_{m \in \text{MOD}}, V)$. Then we inductively define the notion of a formula φ being *satisfied* (or *true*) in $\mathfrak{M}$ at point w as follows (we omit some of the clauses for the booleans):

Semantics

$\mathfrak{M}, w \models p$	iff	$w \in V(p)$,
$\mathfrak{M}, w \models \top$		always,
$\mathfrak{M}, w \models \perp$		never,
$\mathfrak{M}, w \models \neg \varphi$	iff	not $\mathfrak{M}, w \models \varphi$ (notation: $\mathfrak{M}, w \not\models \varphi$),
$\mathfrak{M}, w \models \varphi \wedge \psi$	iff	$\mathfrak{M}, w \models \varphi$ and $\mathfrak{M}, w \models \psi$,
$\mathfrak{M}, w \models \varphi \rightarrow \psi$	iff	$\mathfrak{M}, w \not\models \varphi$ or $\mathfrak{M}, w \models \psi$,
$\mathfrak{M}, w \models \langle m \rangle \varphi$	iff	for some $v \in W$ such that $R^m wv$ we have $\mathfrak{M}, v \models \varphi$,
$\mathfrak{M}, w \models [m] \varphi$	iff	for all $v \in W$ such that $R^m wv$ we have $\mathfrak{M}, v \models \varphi$.

Universal Logical Reasoning in HOL

[SciCompProgr (2019) vol. 172]



Examples for **L** we have already studied:

Modal Logics, Intuitionistic Logics, Free Logic, Description Logics, Conditional Logics, Access Control Logics, Hybrid Logics, Multivalued Logics, Deontic Logics, Provability Logics, Logics with Neighborhood Semantics, Logics with Selection Function Semantics, Paraconsistent Logics, Dynamic Logics, . . .

HOL provers become universal logic reasoning engines!

Universal Logical Reasoning in HOL

Approach: Shallow Semantic Embedding in HOL

(Meta): $s, t ::=$ 

L (target logic): $s, t ::=$ 

Embedding of  in 

Signature of HOL (Constants and Logical Symbols):

	$:=$	
	$:=$	
	$:=$	
	$:=$	

Meta-logical notions:

	$:=$	
---	------	---

Formulas of L are directly identified with terms HOL

Universal Logical Reasoning

Kripke Style Semantics

(propositional modal logic K)

$M, g, s \models P$ if and only if $s \in g(P)$

$M, g, s \models \neg \varphi$ if and only if $M, g, s \not\models \varphi$

$M, g, s \models \varphi \vee \psi$ if and only if $M, g, s \models \varphi$ or $M, g, s \models \psi$

$M, g, s \models \Box \varphi$ if and only if for all t with sRt we have $M, g, t \models \varphi$

Universal Logical Reasoning

Kripke Style Semantics

(propositional modal logic K)

$M, g, s \models P$ if and only if $s \in g(P)$

$M, g, s \models \neg \varphi$ if and only if $M, g, s \not\models \varphi$

$M, g, s \models \varphi \vee \psi$ if and only if $M, g, s \models \varphi$ or $M, g, s \models \psi$

$M, g, s \models \Box \varphi$ if and only if for all t with sRt we have $M, g, t \models \varphi$

Idea: Modal Propositions are mapped to Predicates (= truth-sets) in HOL

- ▶ $P = P_{i \rightarrow o}$
- ▶ $\neg = \lambda \varphi_{i \rightarrow o} \lambda w_i \neg(\varphi w)$
- ▶ $\vee = \lambda \varphi_{i \rightarrow o} \lambda \psi_{i \rightarrow o} \lambda w_i \varphi w \vee \psi w$
- ▶ $\Box = \lambda \varphi_{i \rightarrow o} \lambda w_i \forall v_i R w v \rightarrow \varphi v$

Validity

- ▶ $[\varphi_{i \rightarrow o}] = \forall w_i \varphi w$

Modal Logic (in fact, Hybrid Logic) as a Fragment of HOL

Universal Logical Reasoning

Kripke Style Semantics

(adding quantifiers)

$M, g, s \models \forall x \varphi$ if and only if for all $d \in D$ we have $M, ([d/x]g), s \models \varphi$

This Idea can be extended for Quantifiers

- remember: $\forall x_\alpha s$ is shorthand for $\Pi_{(\alpha \rightarrow o) \rightarrow o}(\lambda x_\alpha s)$ —no binder needed!!!
- $\Pi = \lambda \Phi_{\alpha \rightarrow (i \rightarrow o)} \lambda w_i \Pi_{(\alpha \rightarrow o) \rightarrow o}(\lambda x_\alpha \Phi x w)$

Example (compositionality and λ -conversion at work; omitting types)

$$\begin{aligned}
 [\Box \forall x Px] &\equiv [\Box \overbrace{\Pi(\lambda x Px)}^{\forall x Px}] && \text{def. of } \forall \\
 &\equiv [\Box \overbrace{(\lambda \Phi \lambda w \Pi(\lambda x \Phi x w))(\lambda x Px))}^{\Pi}] && \text{def. of } \Pi \\
 &\equiv [\Box (\lambda w \Pi(\lambda x P x w))] && \beta\text{-reduction} \\
 &\equiv [\overbrace{(\lambda \varphi \lambda w \Pi(\lambda v R w v \rightarrow \varphi v))(\lambda w \Pi(\lambda x P x w))}^{\Box}] && \text{def. of } \Box \\
 &\equiv [\lambda w \forall v (R w v \rightarrow \forall x P x v)] && \beta\text{-reduction} \\
 &\equiv \forall w \forall v (R w v \rightarrow \forall x P x v). && \text{def. of validity, } \beta\text{-reduction}
 \end{aligned}$$

Universal Logical Reasoning

Kripke Style Semantics

(adding quantifiers)

$M, g, s \models \forall x \varphi$ if and only if for all $d \in D$ we have $M, ([d/x]g), s \models \varphi$

This Idea can be extended for Quantifiers

- ▶ remember: $\forall x_\alpha s$ is shorthand for $\Pi_{(\alpha \rightarrow o) \rightarrow o}(\lambda x_\alpha s)$ —no binder needed!!!
- ▶ $\Pi = \lambda \Phi_{\alpha \rightarrow (i \rightarrow o)} \lambda w_i \Pi_{(\alpha \rightarrow o) \rightarrow o}(\lambda x_\alpha \Phi x w)$

Example (compositionality and λ -conversion at work; omitting types)

$$\begin{aligned} [\Box \forall x Px] &\equiv [\Box \overbrace{\Pi(\lambda x Px)}^{\forall x Px}] && \text{def. of } \forall \\ &\equiv [\Box \overbrace{((\lambda \Phi \lambda w \Pi(\lambda x \Phi x w))(\lambda x Px))}^{\Pi}] && \text{def. of } \Pi \\ &\equiv [\Box (\lambda w \Pi(\lambda x Px w))] && \beta\text{-reduction} \\ &\equiv [\overbrace{(\lambda \varphi \lambda w \Pi(\lambda v R w v \rightarrow \varphi v))(\lambda w \Pi(\lambda x Px w))}^{\Box}] && \text{def. of } \Box \\ &\equiv [\lambda w \forall v (R w v \rightarrow \forall x Px v)] && \beta\text{-reduction} \\ &\equiv \forall w \forall v (R w v \rightarrow \forall x Px v). && \text{def. of validity, } \beta\text{-reduction} \end{aligned}$$

Universal Logical Reasoning

Kripke Style Semantics

(adding quantifiers)

$M, g, s \models \forall x \varphi$ if and only if for all $d \in D$ we have $M, ([d/x]g), s \models \varphi$

This Idea can be extended for Quantifiers

- ▶ remember: $\forall x_\alpha s$ is shorthand for $\Pi_{(\alpha \rightarrow o) \rightarrow o}(\lambda x_\alpha s)$ —no binder needed!!!
- ▶ $\Pi = \lambda \Phi_{\alpha \rightarrow (i \rightarrow o)} \lambda w_i \Pi_{(\alpha \rightarrow o) \rightarrow o}(\lambda x_\alpha \Phi x w)$

Example (compositionality and λ -conversion at work; omitting types)

$$\begin{aligned} [\Box \forall x Px] &\equiv [\Box \overbrace{\Pi(\lambda x Px)}^{\forall x Px}] && \text{def. of } \forall \\ &\equiv [\Box \overbrace{((\lambda \Phi \lambda w \Pi(\lambda x \Phi x w))(\lambda x Px))}^{\Pi}] && \text{def. of } \Pi \\ &\equiv [\Box (\lambda w \Pi(\lambda x P x w))] && \beta\text{-reduction} \\ &\equiv [\overbrace{(\lambda \varphi \lambda w \Pi(\lambda v R w v \rightarrow \varphi v))}^{\Box} (\lambda w \Pi(\lambda x P x w))] && \text{def. of } \Box \\ &\equiv [\lambda w \forall v (R w v \rightarrow \forall x P x v)] && \beta\text{-reduction} \\ &\equiv \forall w \forall v (R w v \rightarrow \forall x P x v). && \text{def. of validity, } \beta\text{-reduction} \end{aligned}$$

Universal Logical Reasoning

Kripke Style Semantics

(adding quantifiers)

$M, g, s \models \forall x \varphi$ if and only if for all $d \in D$ we have $M, ([d/x]g), s \models \varphi$

This Idea can be extended for Quantifiers

- remember: $\forall x_\alpha s$ is shorthand for $\Pi_{(\alpha \rightarrow o) \rightarrow o}(\lambda x_\alpha s)$ —no binder needed!!!
- $\Pi = \lambda \Phi_{\alpha \rightarrow (i \rightarrow o)} \lambda w_i \Pi_{(\alpha \rightarrow o) \rightarrow o}(\lambda x_\alpha \Phi x w)$

Example (compositionality and λ -conversion at work; omitting types)

$$\begin{aligned} [\Box \forall x Px] &\equiv [\Box \overbrace{\Pi(\lambda x Px)}^{\forall x Px}] && \text{def. of } \forall \\ &\equiv [\Box \overbrace{((\lambda \Phi \lambda w \Pi(\lambda x \Phi x w))(\lambda x Px))}^{\Pi}] && \text{def. of } \Pi \\ &\equiv [\Box (\lambda w \Pi(\lambda x P x w))] && \beta\text{-reduction} \\ &\equiv [\overbrace{(\lambda \varphi \lambda w \Pi(\lambda v R w v \rightarrow \varphi v))(\lambda w \Pi(\lambda x P x w))}^{\Box}] && \text{def. of } \Box \\ &\equiv [\lambda w \forall v (R w v \rightarrow \forall x P x v)] && \beta\text{-reduction} \\ &\equiv \forall w \forall v (R w v \rightarrow \forall x P x v). && \text{def. of validity, } \beta\text{-reduction} \end{aligned}$$

Universal Logical Reasoning

Kripke Style Semantics

(adding quantifiers)

$M, g, s \models \forall x \varphi$ if and only if for all $d \in D$ we have $M, ([d/x]g), s \models \varphi$

This Idea can be extended for Quantifiers

- remember: $\forall x_\alpha s$ is shorthand for $\Pi_{(\alpha \rightarrow o) \rightarrow o}(\lambda x_\alpha s)$ —no binder needed!!!
- $\Pi = \lambda \Phi_{\alpha \rightarrow (i \rightarrow o)} \lambda w_i \Pi_{(\alpha \rightarrow o) \rightarrow o}(\lambda x_\alpha \Phi x w)$

Example (compositionality and λ -conversion at work; omitting types)

$$\begin{aligned}
 [\Box \forall x Px] &\equiv [\Box \overbrace{\Pi(\lambda x Px)}^{\forall x Px}] && \text{def. of } \forall \\
 &\equiv [\Box \overbrace{((\lambda \Phi \lambda w \Pi(\lambda x \Phi x w))(\lambda x Px))}^{\Pi}] && \text{def. of } \Pi \\
 &\equiv [\Box (\lambda w \Pi(\lambda x Px w))] && \beta\text{-reduction} \\
 &\equiv [\overbrace{(\lambda \varphi \lambda w \Pi(\lambda v R w v \rightarrow \varphi v))(\lambda w \Pi(\lambda x Px w))}^{\Box}] && \text{def. of } \Box \\
 &\equiv [\lambda w \forall v (R w v \rightarrow \forall x Px v)] && \beta\text{-reduction} \\
 &\equiv \forall w \forall v (R w v \rightarrow \forall x Px v). && \text{def. of validity, } \beta\text{-reduction}
 \end{aligned}$$

Universal Logical Reasoning

Kripke Style Semantics

(adding quantifiers)

$M, g, s \models \forall x \varphi$ if and only if for all $d \in D$ we have $M, ([d/x]g), s \models \varphi$

This Idea can be extended for Quantifiers

- remember: $\forall x_\alpha s$ is shorthand for $\Pi_{(\alpha \rightarrow o) \rightarrow o}(\lambda x_\alpha s)$ —no binder needed!!!
- $\Pi = \lambda \Phi_{\alpha \rightarrow (i \rightarrow o)} \lambda w_i \Pi_{(\alpha \rightarrow o) \rightarrow o}(\lambda x_\alpha \Phi x w)$

Example (compositionality and λ -conversion at work; omitting types)

$$\begin{aligned}
 [\Box \forall x Px] &\equiv [\Box \overbrace{\Pi(\lambda x Px)}^{\forall x Px}] && \text{def. of } \forall \\
 &\equiv [\Box \overbrace{((\lambda \Phi \lambda w \Pi(\lambda x \Phi x w))(\lambda x Px))}^{\Pi}] && \text{def. of } \Pi \\
 &\equiv [\Box (\lambda w \Pi(\lambda x P x w))] && \beta\text{-reduction} \\
 &\equiv [\overbrace{(\lambda \varphi \lambda w \Pi(\lambda v R w v \rightarrow \varphi v))(\lambda w \Pi(\lambda x P x w))}^{\Box}] && \text{def. of } \Box \\
 &\equiv [\lambda w \forall v (R w v \rightarrow \forall x P x v)] && \beta\text{-reduction} \\
 &\equiv \forall w \forall v (R w v \rightarrow \forall x P x v). && \text{def. of validity, } \beta\text{-reduction}
 \end{aligned}$$

Universal Logical Reasoning

Kripke Style Semantics

(adding quantifiers)

$M, g, s \models \forall x \varphi$ if and only if for all $d \in D$ we have $M, ([d/x]g), s \models \varphi$

This Idea can be extended for Quantifiers

- ▶ remember: $\forall x_\alpha s$ is shorthand for $\Pi_{(\alpha \rightarrow o) \rightarrow o}(\lambda x_\alpha s)$ —no binder needed!!!
- ▶ $\Pi = \lambda \Phi_{\alpha \rightarrow (i \rightarrow o)} \lambda w_i \Pi_{(\alpha \rightarrow o) \rightarrow o}(\lambda x_\alpha \Phi x w)$

Example (compositionality and λ -conversion at work; omitting types)

$$\begin{aligned} [\Box \forall x Px] &\equiv [\Box \overbrace{\Pi(\lambda x Px)}^{\forall x Px}] && \text{def. of } \forall \\ &\equiv [\Box \overbrace{((\lambda \Phi \lambda w \Pi(\lambda x \Phi x w))(\lambda x Px))}^{\Pi}] && \text{def. of } \Pi \\ &\equiv [\Box (\lambda w \Pi(\lambda x P x w))] && \beta\text{-reduction} \\ &\equiv [\overbrace{(\lambda \varphi \lambda w \Pi(\lambda v R w v \rightarrow \varphi v))(\lambda w \Pi(\lambda x P x w))}^{\Box}] && \text{def. of } \Box \\ &\equiv [\lambda w \forall v (R w v \rightarrow \forall x P x v)] && \beta\text{-reduction} \\ &\equiv \forall w \forall v (R w v \rightarrow \forall x P x v). && \text{def. of validity, } \beta\text{-reduction} \end{aligned}$$

Universal Logical Reasoning

Kripke Style Semantics

(adding quantifiers)

$M, g, s \models \forall x \varphi$ if and only if for all $d \in D$ we have $M, ([d/x]g), s \models \varphi$

This Idea can be extended for Quantifiers

- ▶ remember: $\forall x_\alpha s$ is shorthand for $\Pi_{(\alpha \rightarrow o) \rightarrow o}(\lambda x_\alpha s)$ —no binder needed!!!
- ▶ $\Pi = \lambda \Phi_{\alpha \rightarrow (i \rightarrow o)} \lambda w_i \Pi_{(\alpha \rightarrow o) \rightarrow o}(\lambda x_\alpha \Phi x w)$

Example (compositionality and λ -conversion at work; omitting types)

$$\begin{aligned} [\Box \forall x Px] &\equiv [\Box \overbrace{\Pi(\lambda x Px)}^{\forall x Px}] && \text{def. of } \forall \\ &\equiv [\Box \overbrace{((\lambda \Phi \lambda w \Pi(\lambda x \Phi x w))(\lambda x Px))}^{\Pi}] && \text{def. of } \Pi \\ &\equiv [\Box (\lambda w \Pi(\lambda x P x w))] && \beta\text{-reduction} \\ &\equiv [\overbrace{(\lambda \varphi \lambda w \Pi(\lambda v R w v \rightarrow \varphi v))(\lambda w \Pi(\lambda x P x w))}^{\Box}] && \text{def. of } \Box \\ &\equiv [\lambda w \forall v (R w v \rightarrow \forall x P x v)] && \beta\text{-reduction} \\ &\equiv \forall w \forall v (R w v \rightarrow \forall x P x v). && \text{def. of validity, } \beta\text{-reduction} \end{aligned}$$

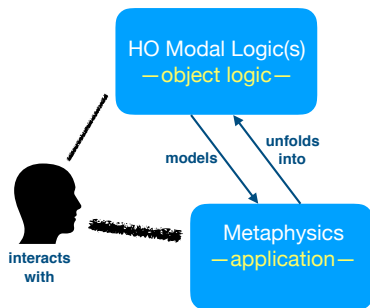
Universal Logical Reasoning in HOL

[SCP, 2019], [OpenPhilos., 2019]



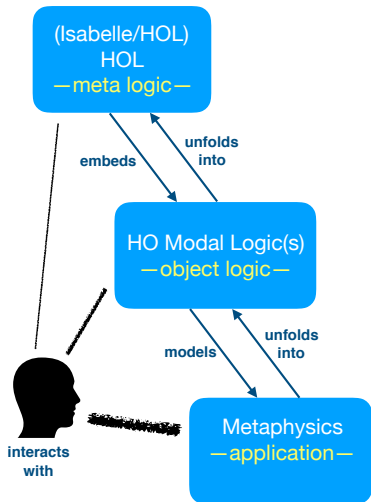
Universal Logical Reasoning in HOL

[SCP, 2019], [OpenPhilos., 2019]



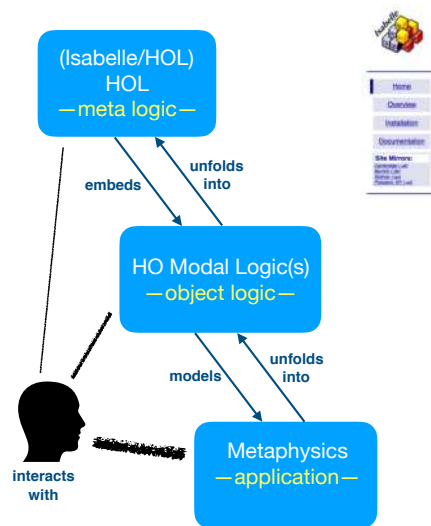
Universal Logical Reasoning in HOL

[SCP, 2019], [OpenPhilos., 2019]



Universal Logical Reasoning in HOL

[SCP, 2019], [OpenPhilos., 2019]



Isabelle



What is Isabelle?

Isabelle is a generic proof assistant. It allows mathematical formulas to be expressed in a formal language and provides tools for proving these formulas in a logical calculus. Isabelle was originally developed at the University of Cambridge and Technische Universität München, but now includes numerous contributions from institutions and individual worldwide. See the [Isabelle 2021 release](#) for a brief introduction.

Now available: Isabelle2017 (October 2017)



[Download for Linux](#) - [Download for Windows \(32bit\)](#) - [Download for Windows \(64bit\)](#) - [Download for Mac OS X](#)

Some notable changes:

- Experimental support for Visual Studio Code as alternative PIDE front-end.
- Improved Isabelle/Edi Power IDE: management of session sources independently of editor buffers, removal of unused theories, explicit indication of theory status, more careful auto-indentation.
- Session-qualified theory imports.
- Code generator improvements: support for statically embedded computations.
- Numerous HOL library improvements.
- More material in HOL-Algebra, HOL-Computational_Algebra and HOL-Analysis (ported from HOL-Light).
- Improved Numsketch model finder, now in main HOL.
- SQL database support in Isabelle/Golem.

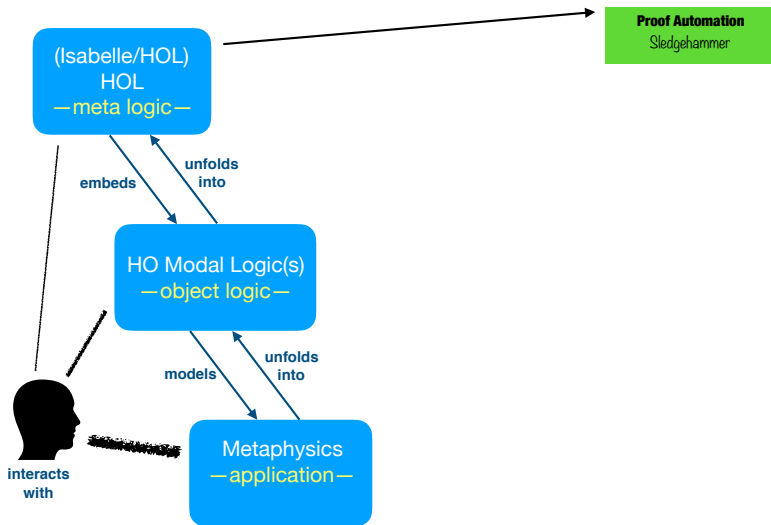
See also the cumulative [NEWS](#).

Distribution & Support

Isabelle is distributed for free under a conglomerate of open-source licenses, but the main code-base is subject to BSD-style regulations. The application bundles include source and binary packages and documentation, see the detailed [Installation Instructions](#). A vast collection of Isabelle examples and applications is available from the [Archive of Formal Proofs](#).

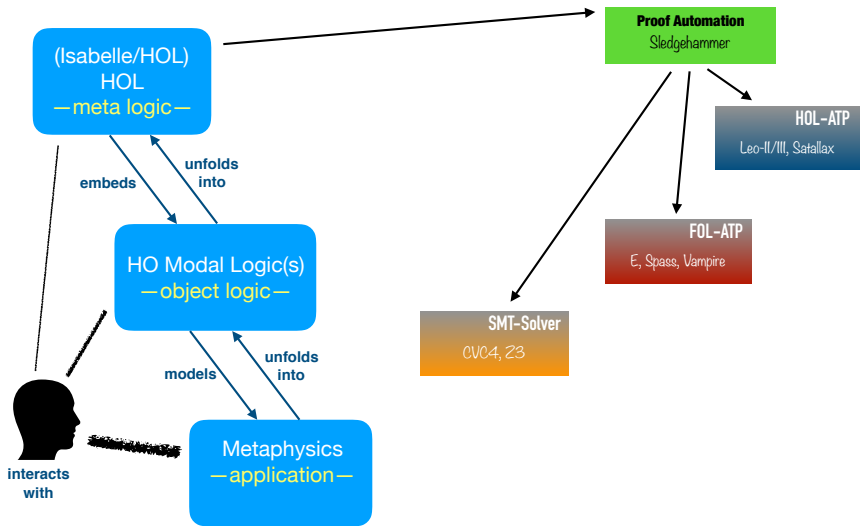
Universal Logical Reasoning in HOL

[SCP, 2019], [OpenPhilos., 2019]



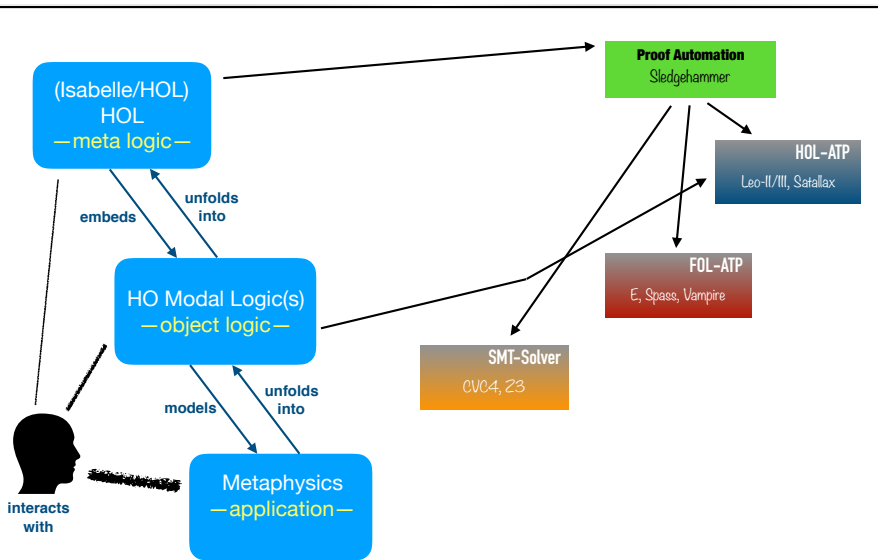
Universal Logical Reasoning in HOL

[SCP, 2019], [OpenPhilos., 2019]



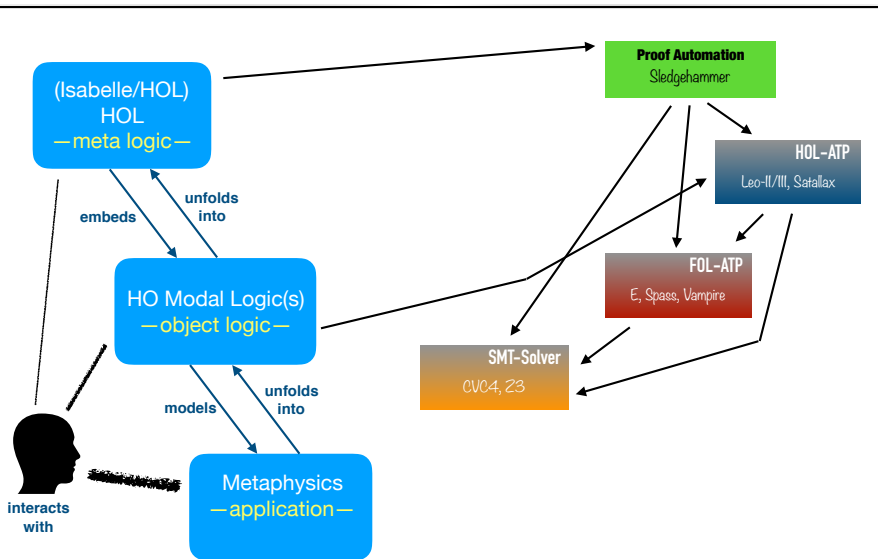
Universal Logical Reasoning in HOL

[SCP, 2019], [OpenPhilos., 2019]



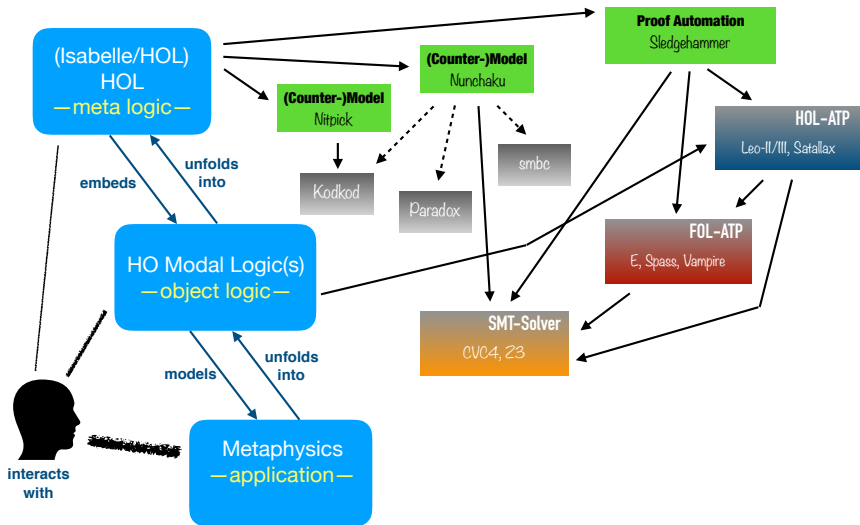
Universal Logical Reasoning in HOL

[SCP, 2019], [OpenPhilos., 2019]



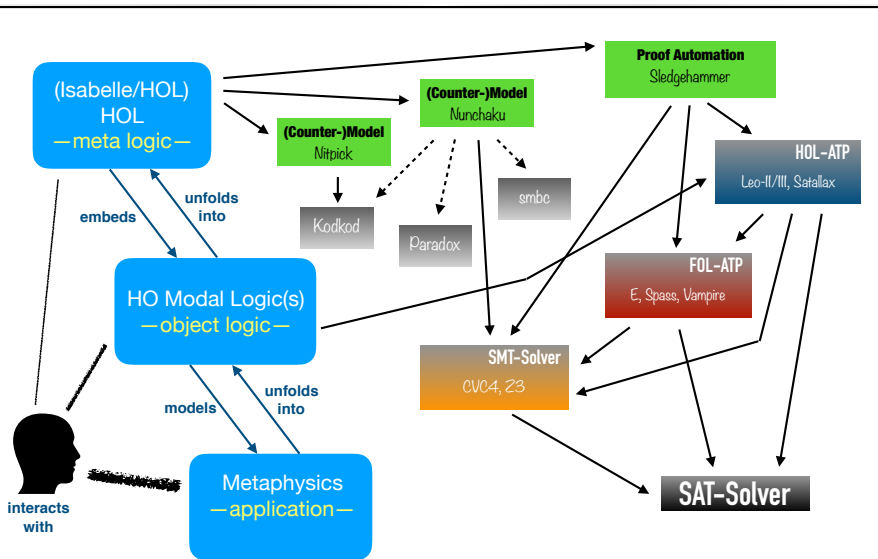
Universal Logical Reasoning in HOL

[SCP, 2019], [OpenPhilos., 2019]



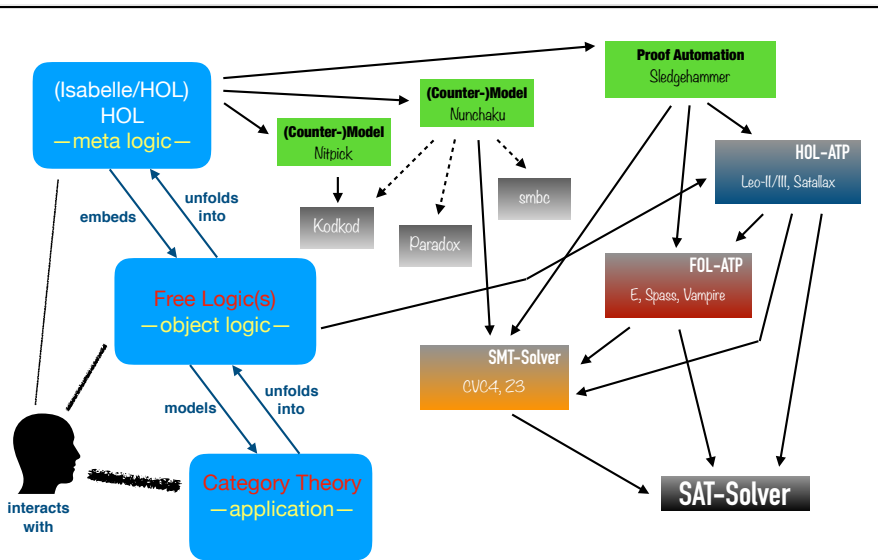
Universal Logical Reasoning in HOL

[SCP, 2019], [OpenPhilos., 2019]



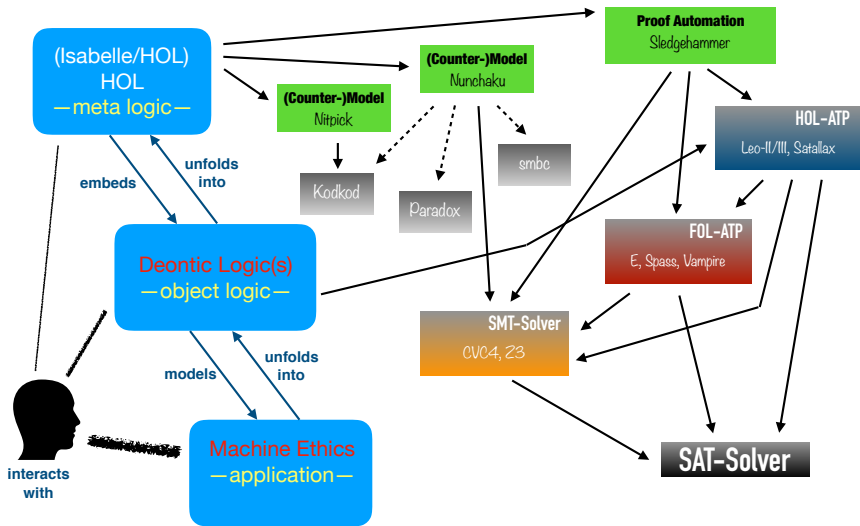
Universal Logical Reasoning in HOL

[SCP, 2019], [OpenPhilos., 2019]



Universal Logical Reasoning in HOL

[SCP, 2019], [OpenPhilos., 2019]

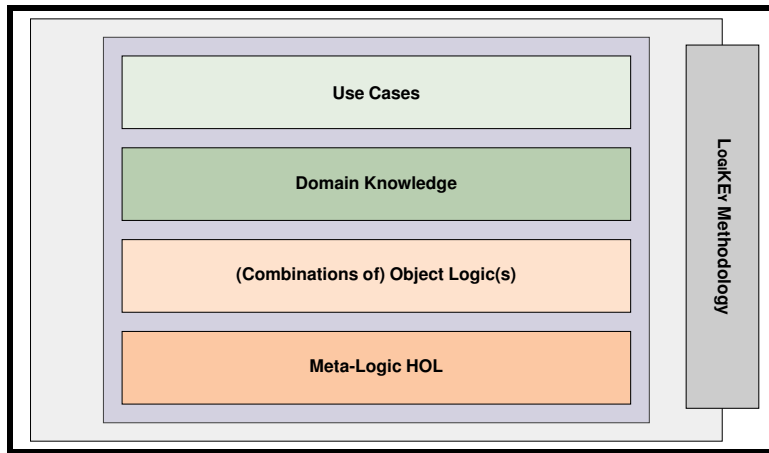




4. LOGiKEY Methodology and Infrastructure for ethical and legal reasoning (jww colleagues from U Luxembourg)

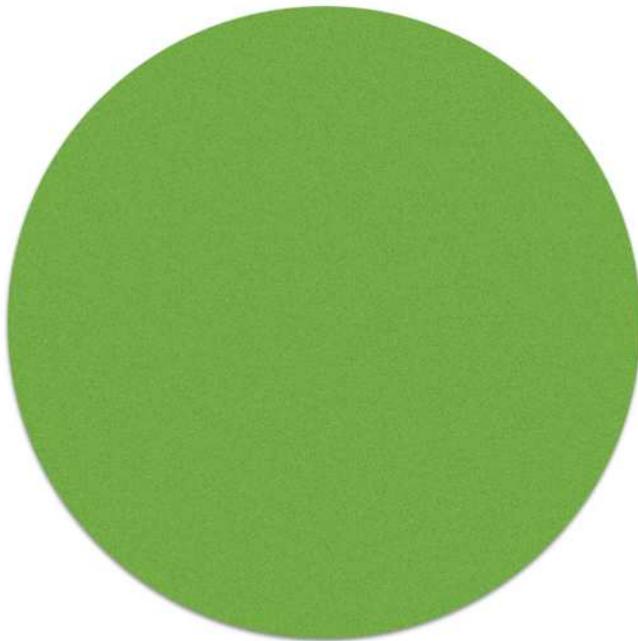
LogiKEy Methodology and Infrastructure

[AIJ (2020) vol. 287], [Data in Brief (2020) vol. 33] and www.logikey.org



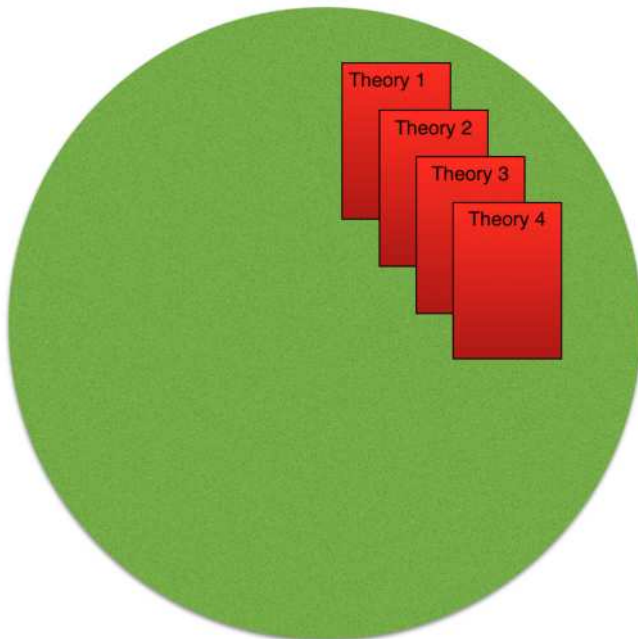
LogiKEy Methodology and Infrastructure

[AIJ (2020) vol. 287], [Data in Brief (2020) vol. 33] and www.logikey.org



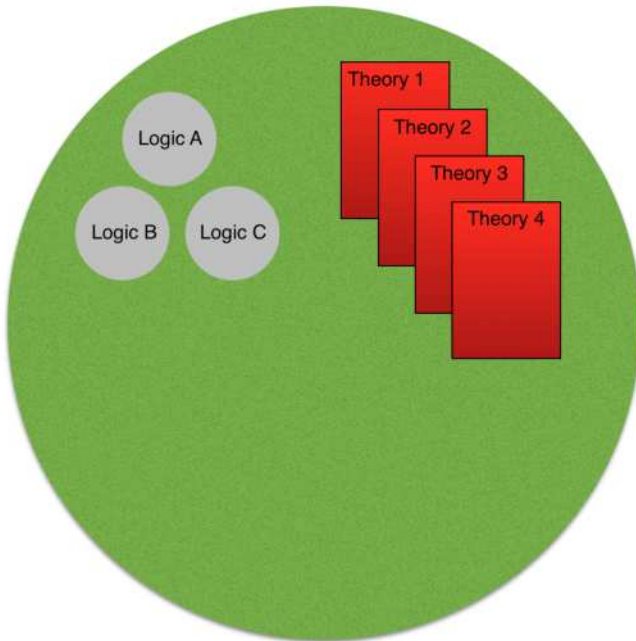
LogiKEy Methodology and Infrastructure

[AIJ (2020) vol. 287], [Data in Brief (2020) vol. 33] and www.logikey.org



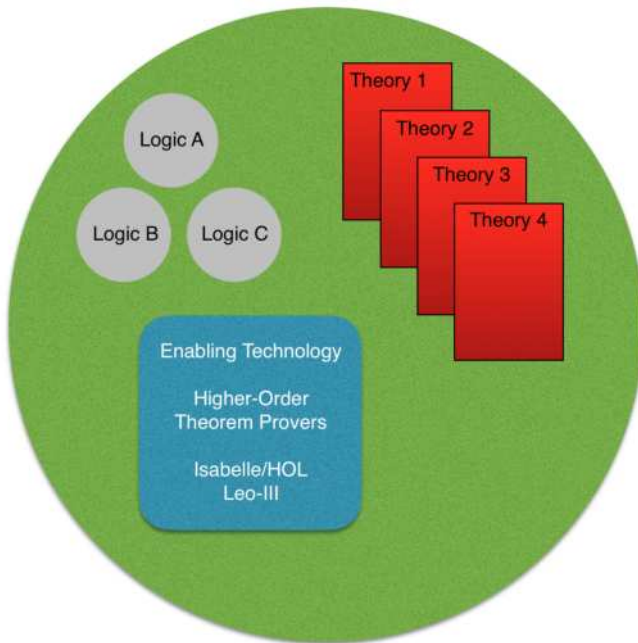
LogiKEy Methodology and Infrastructure

[AIJ (2020) vol. 287], [Data in Brief (2020) vol. 33] and www.logikey.org



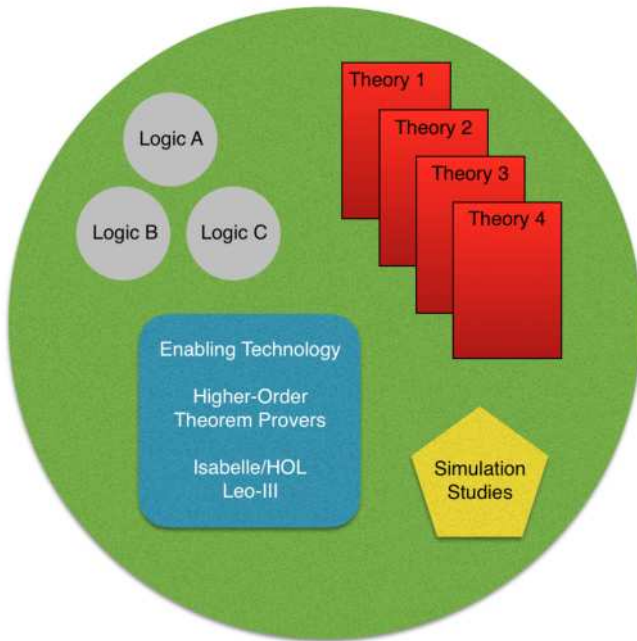
LogiKEy Methodology and Infrastructure

[AIJ (2020) vol. 287], [Data in Brief (2020) vol. 33] and www.logikey.org



LogiKEy Methodology and Infrastructure

[AIJ (2020) vol. 287], [Data in Brief (2020) vol. 33] and www.logikey.org





5. Application to Legal Balancing

Reasoning about **conflicting rights** with
reference to a preference system of **legal values**

(jww D. Fuenmayor and B. Lomfeld)

Application to Legal Balancing

Wild Animal Court Cases — see our recent paper at [MLR-WS@KR (2020)]



Post, a fox hunter, was chasing a fox through public land when Pierson came across the fox and, knowing it was being chased, killed the fox and took it away. Post sued Pierson for damages against his possession of the fox. Post argued that giving chase to the fox was sufficient to establish possession.

- ▶ A local court first ruled in favour of Post; Pierson appealed, decision was changed
- ▶ Decision in favour of Pierson implies: legal **STAB**ility outweighs the **WILL** to possess

STAB > WILL

Application to Legal Balancing

Wild Animal Court Cases — see our recent paper at [MLR-WS@KR (2020)]



Post, a fox hunter, was chasing a fox through public land when Pierson came across the fox and, knowing it was being chased, killed the fox and took it away. Post sued Pierson for damages against his possession of the fox. Post argued that giving chase to the fox was sufficient to establish possession.

- ▶ A local court first ruled in favour of Post; Pierson appealed, decision was changed
- ▶ Decision in favour of Pierson implies: legal **STAB**ility outweighs the **WILL** to possess

STAB > WILL



Chester, a parrot owned by the ASPCA (animal shelter), escaped and was recaptured by Conti. The ASPCA found this out and reclaimed Chester from Conti.

- ▶ Here the court ruled in favour of the ASPCA
- ▶ **STAB > WILL** doesn't apply in this context
- ▶ For a **domestic animal** it is sufficient that the owner did not give up the **RESP**onsability for its maintenance and the **RELI**ance in the animals property

(RESP&RELI) > STAB

Application to Legal Balancing

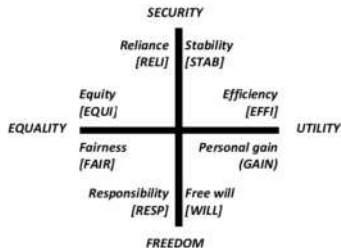
Wild Animal Court Cases — see our recent paper at [MLR-WS@KR (2020)]



Post, a fox hunter, was chasing a fox on public land when Pierson caught it, and, knowing it was being chased, killed the fox and took it away. Post sued Pierson for damages against his possession of the fox. Post argued that giving chase to the fox was sufficient to establish possession.

- ▶ A local court first ruled in favour of Post; Pierson appealed, decision was changed
- ▶ Decision in favour of Pierson implies: legal **STAB**ility outweighs the **WILL** to possess

STAB > WILL



Value Ontology [Lomfeld (2019)]



owned by the ASPCA (animal was captured by Conti. This out and reclaimed Chester from Conti.

- ▶ Here the court ruled in favour of the ASPCA
- ▶ **STAB** > **WILL** doesn't apply in this context
- ▶ For a **domestic animal** it is sufficient that the owner did not give up the **RESP**onsability for its maintenance and the **RELI**ance in the animals property

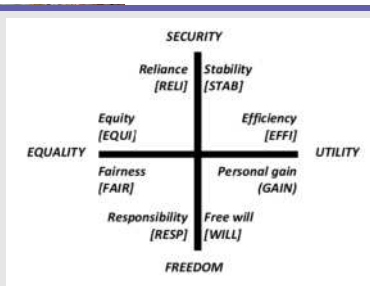
(RESP&RELI) > STAB

Application to Legal Balancing

Wild Animal Court Cases — see our recent paper at [MLR-WS@KR (2020)]



Post, a fox hunter, was chased off public land when Pierson caught the fox, and, knowing it was his, took it away. Post brought an action against his possession, claiming that giving chase to the fox established possession.



Value Ontology [Lomfeld (2019)]



owned by the ASPCA (animal was captured by Conti and was recaptured by Conti. This out and reclaimed Chester

```
(*LWK: (prima facie) value preferences for kinds of situations*)
axiomatization where
R1: "[appAnimal → ([STAB*] < [STAB*])]" and
R2: "[appWildAnimal → ([WILL*] < [STAB*])]" and
R3: "[appDomAnimal → ([STAB*] < [RELI*RESP*])]"
```

- ▶ A local court first ruled in favour of Post; Pierson appealed, decision was changed
- ▶ Decision in favour of Pierson implies: legal **STAB**ility outweighs the **WILL** to possess

STAB > WILL

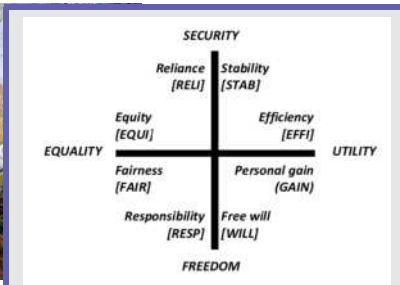
- ▶ In favour of the ASPCA
- ▶ apply in this context

For a **domestic animal** it is sufficient that the owner did not give up the **RESP**onsibility for its maintenance and the **RELI**ance in the animals property

(RESP&RELI) > STAB

Application to Legal Balancing

Wild Animal Court Cases — see our recent paper at [MLR-WS@KR (2020)]



Post, a fox hunter, was chasing a fox on public land when Pierson caught the fox and, knowing it was his property, took it away. Post brought an action against his possession, claiming that giving chase to the fox established possession.

Value Ontology [Lomfeld (2019)]

owned by the ASPCA (animal was captured by Conti. Pierson was out and reclaimed Chester

```
(*LWK: (prima facie) value preferences for kinds of situations*)
axiomatization where
R1: "[appAnimal → ([STAB*] < [STAB*])]" and
R2: "[appWildAnimal → ([WILL*] < [STAB*])]" and
R3: "[appDomAnimal → ([STAB*] < [RELI*RESP*])]"
```

```
(*decision for plaintiff (Post) can be proven automatically*)
theorem Post: "p_theory → [p_facts → □-For p]"
by (smt F1 F3 ForAx tBR SBR def other.simps)

(*decision for defendant (Pierson) can be proven automatically*)
theorem Pierson: "d_theory → [d_facts → □-For d]"
by (smt F1 F3 ForAx R2 W5 W7 other.simps tSBR)

(*decision for plaintiff (ASPCA) can be proven automatically*)
theorem ASPCA: "p_theory → [p_facts → □-For p]"
by (smt F3 F4 F5 ForAx R3 W6 W8 tBR SBR_def other.simps(1))
```

- ▶ A local court first ruled in favour of Pierson appealed.
- ▶ Decision in favour of Post, STABILITY outweighed FREEDOM.

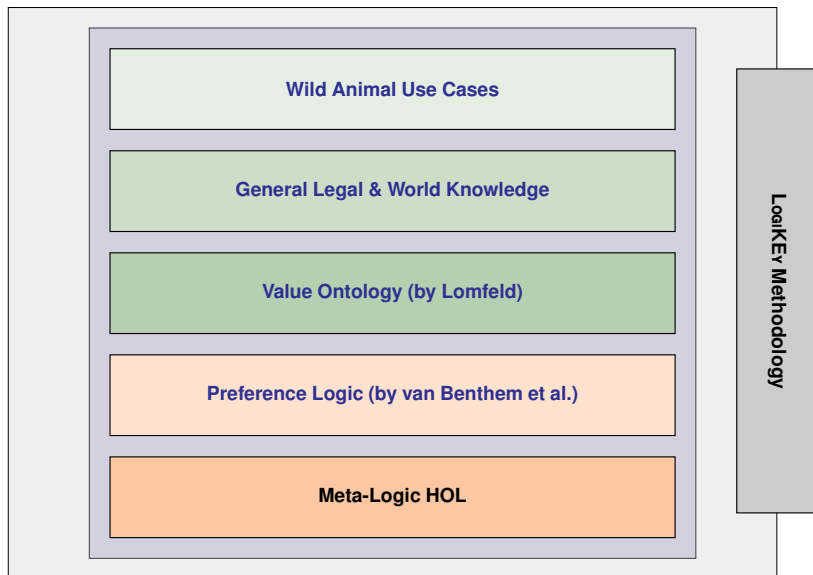
STAB

favour of the ASPCA
apply in this context

and it is sufficient that the
the **RESP**onsibility for
the **RELI**ance in the

> STAB

Adaptation of LogiKEy



Legal Balancing: The Legal Argument for Pierson

```
1 theory Pierson imports GeneralKnowledge (** Benzmüller, Fuennayor & Lomfeld, 2021 **)
2 begin (** Pierson v. Post "wild animal" case **)
3   (*unimportant*) nitpick_params[user_axioms,expect=genuine,show_all,format=3]
4   (*case-specific 'world-vocabulary'*)
5   consts α::"e" (*appropriated animal (fox in this case) *)
6   consts Pursue::"c⇒e⇒α" Capture::"c⇒e⇒α"
7   (****** pro-defendant (Pierson) argument ******)
8   (*defendant's theory*)
9   abbreviation "dT1 ≡ {(∃c. Capture c α ∧ ¬Domestic α) → appWildAnimal}"
10  abbreviation "dT2 ≡ {∀c. Pursue c α → Intent c}"
11  abbreviation "dT3 ≡ {∀c. Capture c α → Poss c}"
12  abbreviation "d_theory ≡ dT1 ∧ dT2 ∧ dT3"
13  (*defendant's facts*)
14  abbreviation "dF1 w ≡ Fox α w"
15  abbreviation "dF2 w ≡ FreeRoaming α w"
16  abbreviation "dF3 w ≡ ¬Pet α w"
17  abbreviation "dF4 w ≡ Pursue p α w"
18  abbreviation "dF5 w ≡ Capture d α w"
19  abbreviation "d_facts ≡ dF1 ∧ dF2 ∧ dF3 ∧ dF4 ∧ dF5"
20  (*decision for defendant (Pierson) can be proven automatically*)
21  theorem Pierson: "d_theory → [d_facts → □-For d]" sledgehammer[verbose]
22    by (smt F1 F3 ForAx R2 W5 W7 other.simps tSBR)
23  (*we reconstruct the reasoning process leading to the decision for the defendant*)
24  theorem Pierson': assumes d_theory and "d_facts w" shows "□-For d w"
25  proof -
26    have 1: "appWildAnimal w" using W5 W7 asms by blast
27    have 2: "[WILL] < [STAB]" using 1 R2 asms by fastforce
28    have 3: "[□-WILL] → □-[STAB]" using 2 tSBR by smt
29    have 4: "□-(For p ↔ □-[WILL]) w" using F1 asms by meson
30    have 5: "□-(For d ↔ □-[STAB]) w" using F3 asms by meson
31    have 6: "□-(□-[WILL] ∨ □-[STAB]) w" using 4 5 ForAx by (smt other.simps)
32    have 7: "□-(□-[STAB]) w" using 3 6 by blast
33    have 8: "□-(For d) w" using 5 7 by simp
34    then show ?thesis by simp
35  qed
36  (****** Further checks (using model finder) ******)
37  (*defendant's theory and facts are logically consistent*)
38  lemma "d_theory ∧ [d_facts]" nitpick[satisfy,card =3] oops (* (non-trivial) model found*)
39  (*decision for defendant is compatible with premises and lacks value conflicts*)
40  lemma "[¬Conflict] ∧ [¬Conflict] ∧ d_theory ∧ [d_facts A For d]"
41    nitpick[satisfy,card =3] oops (* (non-trivial) model found*)
42  (*situations where decision holds for plaintiff are compatible too*)
43  lemma "[¬Conflict] ∧ [¬Conflict] ∧ d_theory ∧ [d_facts A For p]"
44    nitpick[satisfy,card =3] oops (* (non-trivial) model found*)
45 end
```

Ask for Demo

Pierson's theory & facts

Pierson's argument automated

Pierson's argument step by step

Various checks:
consistency,
non-conflicting,

...

Legal Balancing: The Legal Argument for Conti

```
1 theory Conti imports GeneralKnowledge (** Benz Müller, Fuernmayor & Lonfeld, 2021 **)
2 begin (** Conti v. ASPCA "wild animal" case **)
3   (*unimportant*) nitpick_params[user_axioms,expect=genuine,show_all,format=3]
4   (*case-specific 'world-vocabulary'*)
5   consts o::"e" (*appropriated animal (parrot in this case) *)
6   consts Care::"C⇒e⇒e" Prop::"C⇒e⇒e" Capture::"C⇒e⇒e"
7   (*===== pro-plaintiff (ASPCA) argument =====*)
8   (*plaintiff's theory*)
9   abbreviation pT1 ≡ [(∃c. Capture c o A Domestic o) → appDomAnimal]
10  abbreviation pT2 ≡ [∀c. Care c o → Mtn c]
11  abbreviation pT3 ≡ [∀c. Prop c o → Own c]
12  abbreviation pT4 ≡ [∀c. Capture c o → Poss c] (*concedes' to defendant*)
13  abbreviation "p_theory" ≡ pT1 ∧ pT2 ∧ pT3 ∧ pT4
14  (*plaintiff's facts*)
15  abbreviation "pF1" ≡ Parrot o w
16  abbreviation "pF2" ≡ Pet o w
17  abbreviation "pF3" ≡ Care p o w
18  abbreviation "pF4" ≡ Prop p o w
19  abbreviation "pF5" ≡ Capture d o w
20  abbreviation "p_facts" ≡ pF1 ∧ pF2 ∧ pF3 ∧ pF4 ∧ pF5
21  (*decision for plaintiff (ASPCA) can be proven automatically*)
22  theorem ASPCA: "p_theory ⟹ [p_facts ⟹ □For p]"
23    by (smt F3 F4 F5 ForAx R3 W6 W8 tBR SBR_def other.simps(1))
24  (*we reconstruct the reasoning process leading to the decision for the plaintiff*)
25  theorem ASPCA': assumes p_theory and "p_facts w" shows "□For p w"
26  proof -
27    have 1: "appDomAnimal w" using W6 W8 asses by blast
28    have 2: "[([STAB] ↪ [RELI⇒RESP])]" using 1 R3 by fastforce
29    have 3: "[([STAB])] → □([RELI] V [RESP])" using 2 tSBR by smt
30    have 4: "□(For p ⟹ □([RELI]) w)" using F5 asses by metis
31    have 5: "□(For p ⟹ □([RESP]) w)" using F4 asses by metis
32    have 6: "□(For d ⟹ □([STAB]) w)" using F3 asses by meson
33    have 7: "□((□([RELI]) V (□([RESP]) V (□([STAB])))) w"
34      using 4 5 6 ForAx by (smt other.simps)
35    have 8: "□((□([RELI]) V (□([RESP])) w" using 3 7 by metis
36    have 9: "□(For p) w" using 4 5 8 by auto
37    then show ?thesis by simp
38  qed
39  (*===== Further checks (using model finder) =====*)
40  (*plaintiff's theory and facts are logically consistent*)
41  lemma "p_theory ∧ [p_facts]" nitpick[satisfy,card =3] oops (* (non-trivial) model found*)
42  (*decision for plaintiff is compatible with premises and lacks value conflicts*)
43  lemma "[¬Conflict] ∧ [¬Conflict] ∧ p_theory ∧ [p_facts A For p]"
44    nitpick[satisfy,card =3] oops (* (non-trivial) model found*)
45  (*situations where decision holds for defendant are compatible too*)
46  lemma "[¬Conflict] ∧ [¬Conflict] ∧ p_theory ∧ [p_facts A For d]"
47    nitpick[satisfy,card =3] oops (* (non-trivial) model found*)
48 end
```

Conti's theory & facts

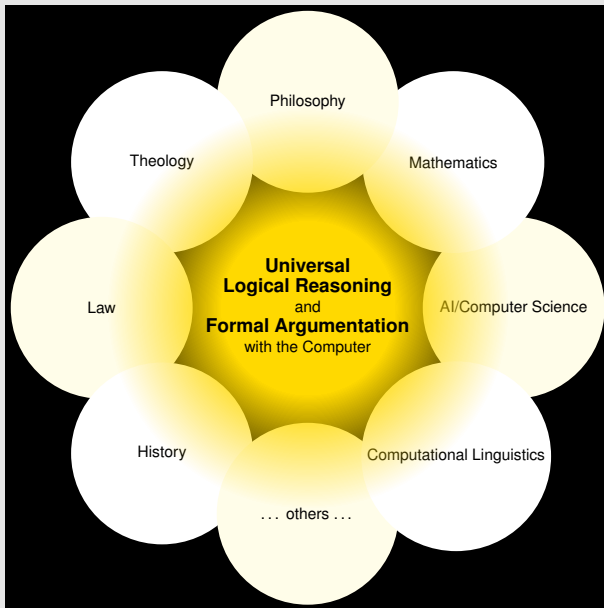
Ask for Demo

Conti's argument
automated

Conti's argument
step by step

Various checks:
consistency,
non-conflicting,
...

Conclusion & Outlook



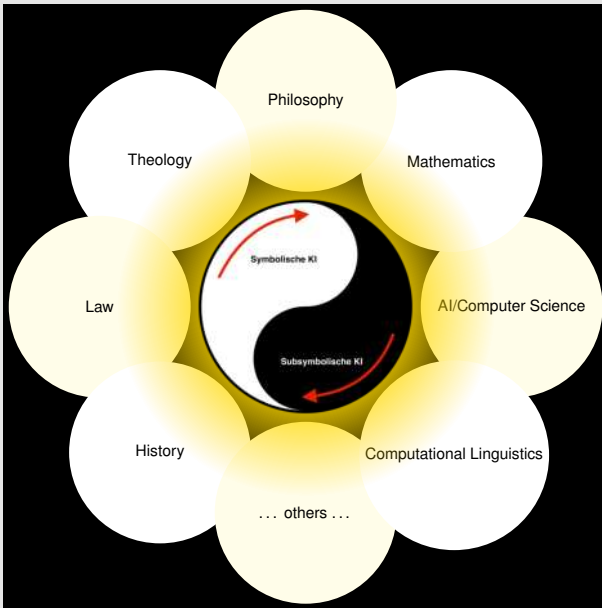
Exploitation:

- ▶ research
- ▶ **education**
- ▶ application

Ongoing & next steps:

- ▶ joint projects (research)
- ▶ new textbook & courses (education)
- ▶ collaborations with startups and industry

Conclusion & Outlook



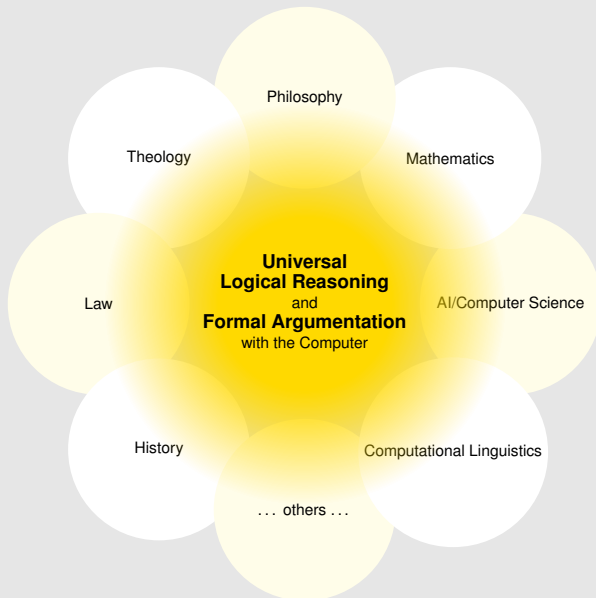
Exploitation:

- ▶ research
- ▶ **education**
- ▶ application

Ongoing & next steps:

- ▶ joint projects (research)
- ▶ new textbook & courses (education)
- ▶ collaborations with startups and industry

Conclusion & Outlook



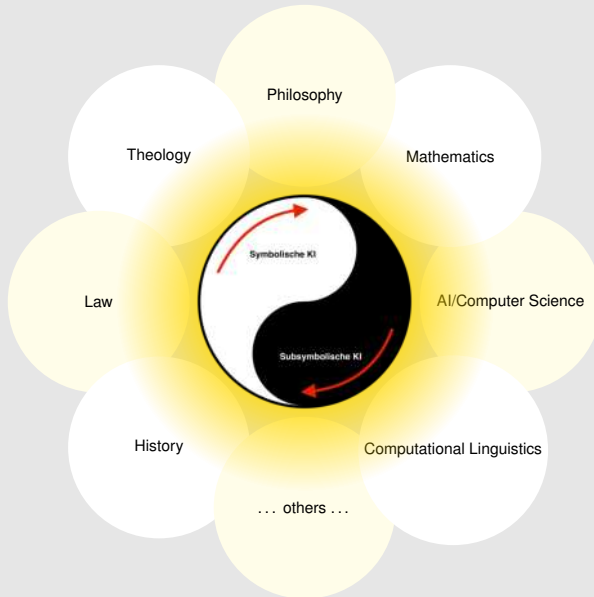
Exploitation:

- ▶ research
- ▶ **education**
- ▶ application

Ongoing & next steps:

- ▶ joint projects (research)
- ▶ new textbook & courses (education)
- ▶ collaborations with startups and industry

Conclusion & Outlook



Exploitation:

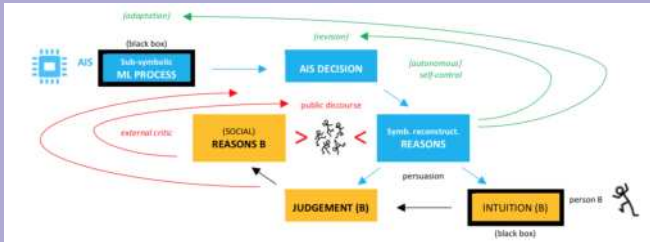
- ▶ research
- ▶ **education**
- ▶ application

Ongoing & next steps:

- ▶ joint projects (research)
- ▶ new textbook & courses (education)
- ▶ collaborations with startups and industry

Outlook: Towards Reasonable Machines

Artificial “Social Reasoning Model”

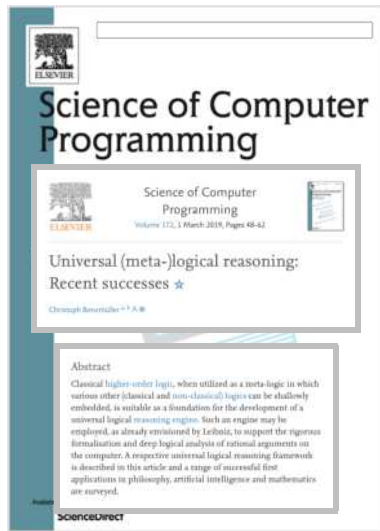
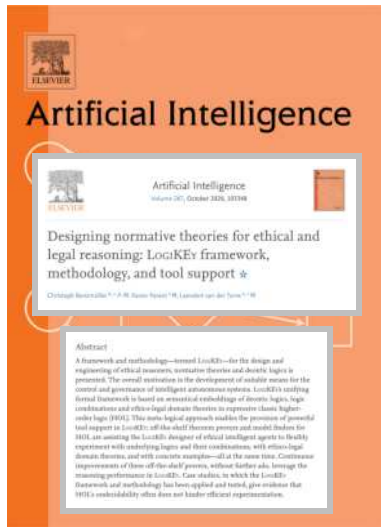


Building trust into AI systems through rational communication of »reasons«

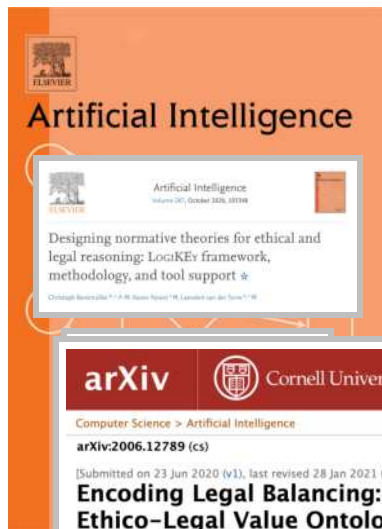
Important:

- ▶ these reasons may actually be independent of the original motivational impulse to act
- ▶ should be possible to avoid opening the black-box, since this can increase vulnerability

Main References for this Talk



Main References for this Talk



REASONABLE MACHINES: A RESEARCH MANIFESTO

BERNHARD BENZMÜLLER & BERTRAM LOMFELD

Dep. of Mathematics and Computer Science & Dep. of Law, FU Berlin

Freie Universität



Berlin

ABSTRACT

Future intelligent autonomous systems (IAS) are inevitably deciding on moral and legal questions, e.g. in self-driving cars, health care or human-machine collaboration. As decision processes in most modern sub-symbolic IAS are hidden, the simple political plea for transparency, accountability and governance falls short. A sound ecosystem of trust requires ways for IAS to autonomously justify their actions, that is, to learn giving and taking reasons for their decisions. Building on social reasoning models in moral psychology and legal philosophy such an idea of »REASONABLE MACHINES« requires novel, hybrid reasoning tools, ethico-legal ontologies and associated argumentation technology. Enabling machines to normative communication creates trust and opens new dimensions of AI application and human-machine interaction.

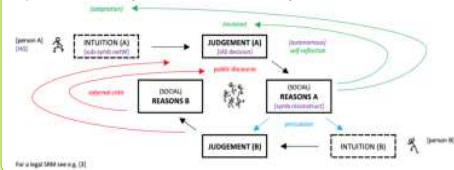
CORE OBJECTIVES

- enabling argument-based explanations & justifications of IAS decisions,
- enabling ethico-legal reasoning about, and public critique of, IAS decisions,
- facilitating political and legal governance of IAS decision making,
- evolving ethico-legal agency and communicative capacity of IASs,
- enabling trustworthy human-interaction by normative communication,
- fostering development of novel neuro-symbolic AI architectures.

Long-term vision: To enable machines to give and take normative reasons for their decisions and actions capacitates them to engage in communicative action within social systems.

[ARTIFICIAL] SOCIAL REASONING MECHANISM ([A]SRM)

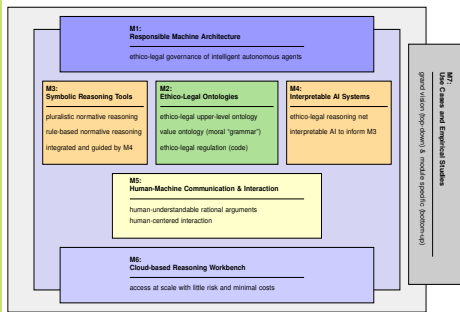
The parallel to human SRM guides the overall architectural design of REASONABLE MACHINES.



REFERENCES

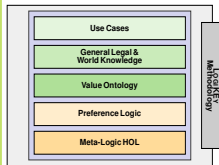
- [1] BENZMÜLLER, C., FUENMAYOR, D., AND LOMFELD, B. Encoding legal balancing: Automating an abstract ethico-legal value ontology in preference logic. In *WS on Models of Legal Reasoning, hosted by KR* (2020).
- [2] BENZMÜLLER, C., PARENT, X., AND VAN DER TORRE, L. Designing normative theories for ethical and legal reasoning: Logikey framework, methodology, and tool support. *Artificial Intelligence* 287 (2020), 103348.
- [3] LOMFELD, B. Emotio iuris. Skizzen zu einer psychologisch aufgeklärten Methodenlehre des Rechts. In *Recht Fühlen* (2017), Köhler, Müller-Mall, Schmidt, and Schnädelbach, Eds., Fink, München, pp. 19–32.
- [4] LOMFELD, B. Grammatik der Rechtfertigung: Eine kritische Rekonstruktion der Rechts(fort)bildung. *Kritische Justiz* 52, 4 (2019).

MODULAR STRUCTURE OF REASONABLE MACHINES RESEARCH



LOGIKEY METHODOLOGY

LOGIKEY [2] is a framework & methodology for the design and engineering of ethical reasoners, normative theories and deontic logics. For recent formalization work see [1] and logikey.org.

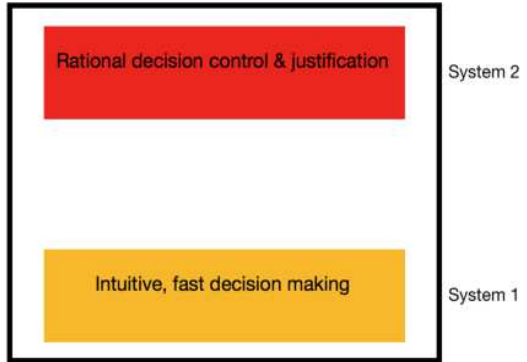
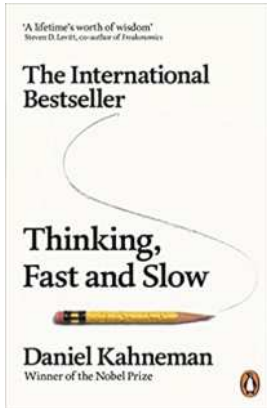


VALUE BASED EXPLANATIONS

The reflexive symbolic/sub-symbolic feedback loop uses value categories based on an established discursive moral grammar scheme [4], which we already encoded for legal balancing [1].

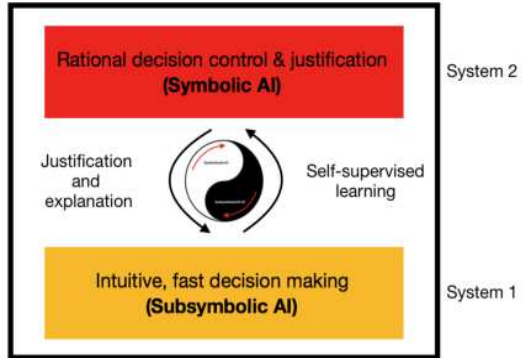
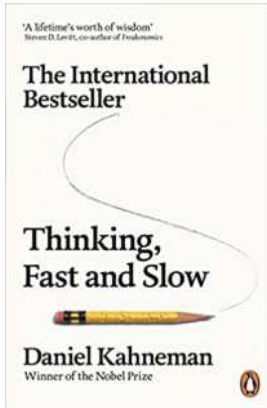


How to create trust? How to control?



See e.g. also: **J. Haidt, The Emotional Dog and its Rational Tail, 2001**

How to create trust? How to control?



See e.g. also: **J. Haidt, The Emotional Dog and its Rational Tail, 2001**

Scientific Profile

Theory & Education

Publications

≥ 40 Journal Articles, ≥ 20 Book Chapters,
≥ 130 Conf./WS Papers, ≥ 15 Proceedings

Research Grants

5xDFG, BMBF, ERC, Volkswagenstiftung,
Heisenberg Fellow, Studienstiftung (PhD)

Keynotes & Invited Talks

≥ 100 at Intl. Conferences/WSs, Institutes

Courses & Seminars

≥ 50: GER (Berlin, Saarbr., Darmst.),
USA, CHN, LUX, SUI, DK, FRA, BRA, AUS

Current Student Supervision

≥ 7 PhD (GER/LUX), ≥ 10 MSc/BSc

Tools, Applications & Industry

Theorem Provers

Leo-I: Pioneered TPTP THF
Leo-II: World Champion in 2010
Leo-III: World Champion in 2019

Industry Engagements & Cooperations

Germany: Latentine GmbH, DFKI GmbH
Poland/UK: BILLON SP. Z O.O.
USA: Articulate Software
Luxembourg: KPMG Lighthouse
Liechtenstein: IDNI AG

Community Service

Organizations

CLAIRE: natl. contact point, CADE: VP,
AIQT, CICM, DEON, GI-KI, etc.: committee
LJ.IGPL, J.Appl.Log, Hist.Log.: edit. board

Stations: Saarbr., Pittsburgh, Edinburgh, Cambridge, Sacramento, Berlin, Stanford, Luxemb.